NPLify · P0 Technical Baseline

Deliverable 1 — Entity-Relationship Diagram & Data Model · Draft v5.7

New XP Technologies Limited

11 October 2026 · Confidential · Internal baseline document

Status and conventions

Status: Draft v5.7 (11 October 2026) is v5.6 plus D46, the ledger design agreed with NPL on 10–11 October (variance by cause, loss booked to NPL first, accruals at conversion, NPL-GR's two invoices with the remittance markup absorbing partner cost and forex, earnings kept unconverted and offset between partners, the promise discharged on release, fee parts on the principal): two tables, three views, invariants 40–44. Draft v5.6 (9 October 2026) was v5.5 with personal names removed from every example and attribution; v5.5 closed the 8–9 October work, consolidating v5.3 and v5.4 and carrying every decision to D45 — a CLIENT parent above PROJECT with a client-level roll-up (D38), NPL's operational answers (D39–D41), the quote-validity extensions and the honoured expired rate (D45, replacing D42), the reporting currency and one system-settings area with ROLE and ROLE_RIGHT (D43, D44). It applies the thirteen fee-practice decisions D20–D32, the same-currency rule D33 the three FRS-term decisions D34–D36 and the naming alignment D37, confirmed by NPL on 8–9 October from the fee table, NPL's workbook answers and the Project Context and Payment methods tabs: volume tiers and per-sender splits (D20, D21), the customer-facing fee (D22), NPL-GR as an own-desk project (D23), invoices paid in parts (D24), a rounding menu (D25), markup shared with a client party and rebates as their own stream (D26, D27), partner rates as quoted (D28), the bank cutoff and rate lock (D29), cash by token with rates per city (D30), wallet screening (D31), single-use accounts and loss events (D32), and same-currency pass-through (D33). Fourteen tables and five views are added; invariants 3 and 9 are restated; invariants 20–39 are new. NPL's answers to the operational questions of 9 October are applied as D39–D41, the quote-validity extensions and the honoured expired rate as D45 (replacing D42), the system reporting currency as D43 and the single system-settings area (roles, settings, platform pairs) as D44. The amendments of 9 October (D15 closed, D27, D29) the evening additions D34–D36 (holiday calendar, quote package, message template — the FRS terms that had no home) and the naming alignment D37 are recorded in §6. It carries v5.0–v5.2 (D13–D19) unchanged. Supersedes v5.5, v5.4 and v5.3 (the working drafts of 8–9 October), v5.2, v5.1, v5.0 and v4.0 in full. This baseline is an internal New XP document; the client deliverables are derived from it. The Ledger Posting Design and its Reading Companion are regenerated with this version (numbered v5.7); the other dependent documents are listed in §6.

Convention: all monetary amounts are stored as exact decimals; every table carries `created_at` / `created_by`; audit, approvals and locking are modelled once (domain 4) and apply to all master data. Derived figures (client balance, group entitlement, offsets, custody) are **views over postings and deals — never stored**.

Seven diagrams across four domains, one schema: 70 tables and 12 views. In each diagram, entities defined in another diagram appear with no attributes and name that diagram. Relationship lines run from the referenced row to the row that stores the foreign key; `||--o{` is composition (the parent owns the child), `||..o{` a reference or a polymorphic link resolved by a type column plus an id. Diagram 3a of v5.2 is split in v5.3 because the deal leg and its stamps outgrew one page: 3a is the deal and its collection, 3b the conversion and its exceptions, and the settlement diagram (3b in v5.2) becomes 3c. Section 6 lists what changed since v4.0; Appendix A reconciles the model with the FRS; Appendix B is the data dictionary.

---

1 · Configuration domain

A **Client** is the commercial relationship NPL contracts with; a **Project** is one arrangement with it (D38) and owns its configuration. Partners are global counterparties; how a partner is used *in this project* is the **partner configuration** block — fees, vehicles, endpoints, destination approvals and rebate terms hang off it. Counterparty receivers are organised in **receiver groups**, the unit entitlement is tracked on, and each receiving entity holds one or more **accounts** with a currency and rail.

Diagram 1a. `PARTNER_CONFIG`, `PARTNER_PAIR`, `DESTINATION_APPROVAL`, `FEE_STRUCTURE` are defined in Diagram 1b.

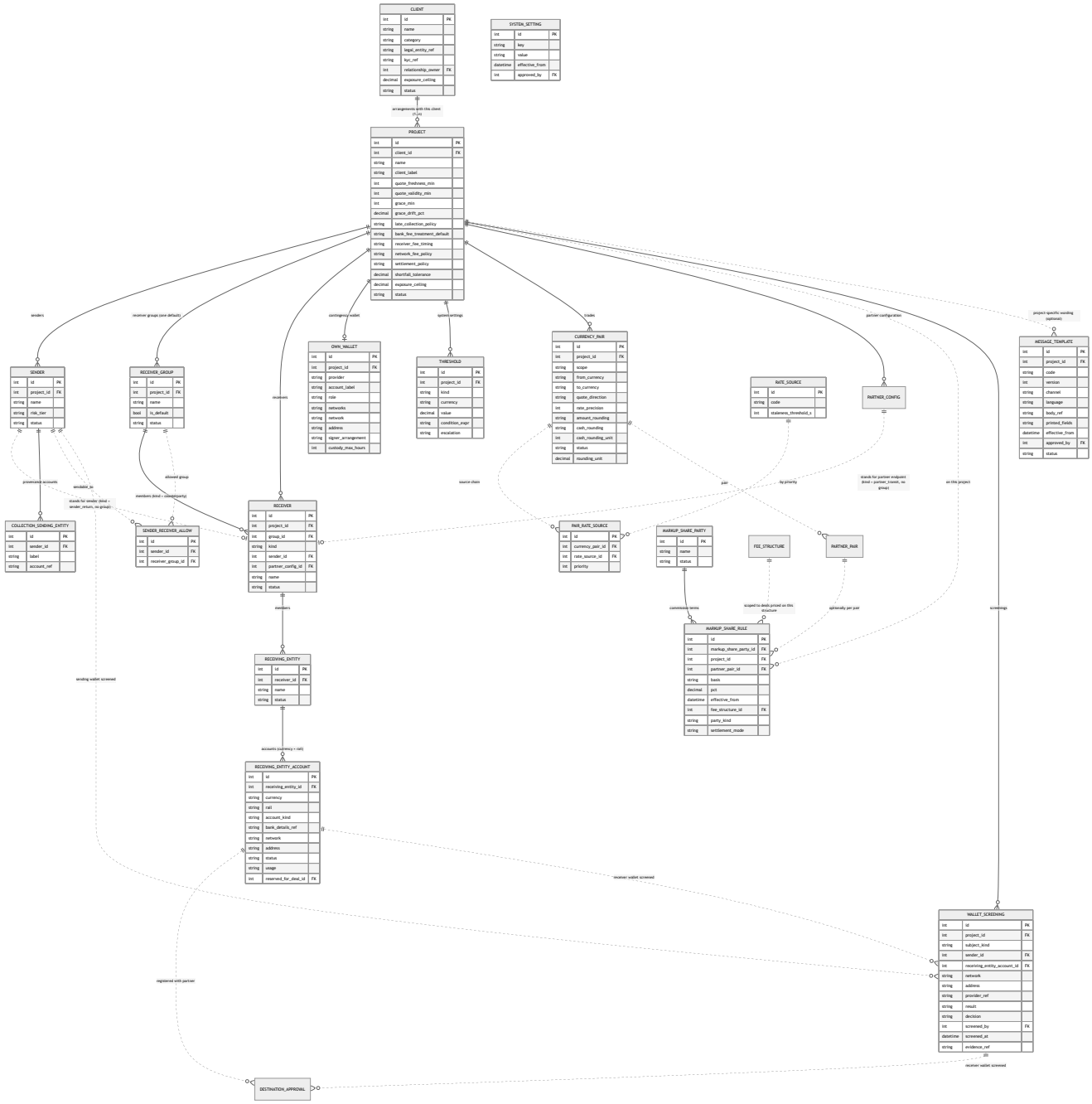


Diagram 1a — Configuration — project, parties, receiver groups and wallet screening

Diagram 1b. `PROJECT`, `CURRENCY_PAIR`, `SENDER`, `RECEIVING_ENTITY_ACCOUNT`, `OWN_WALLET` are defined in Diagram 1a.

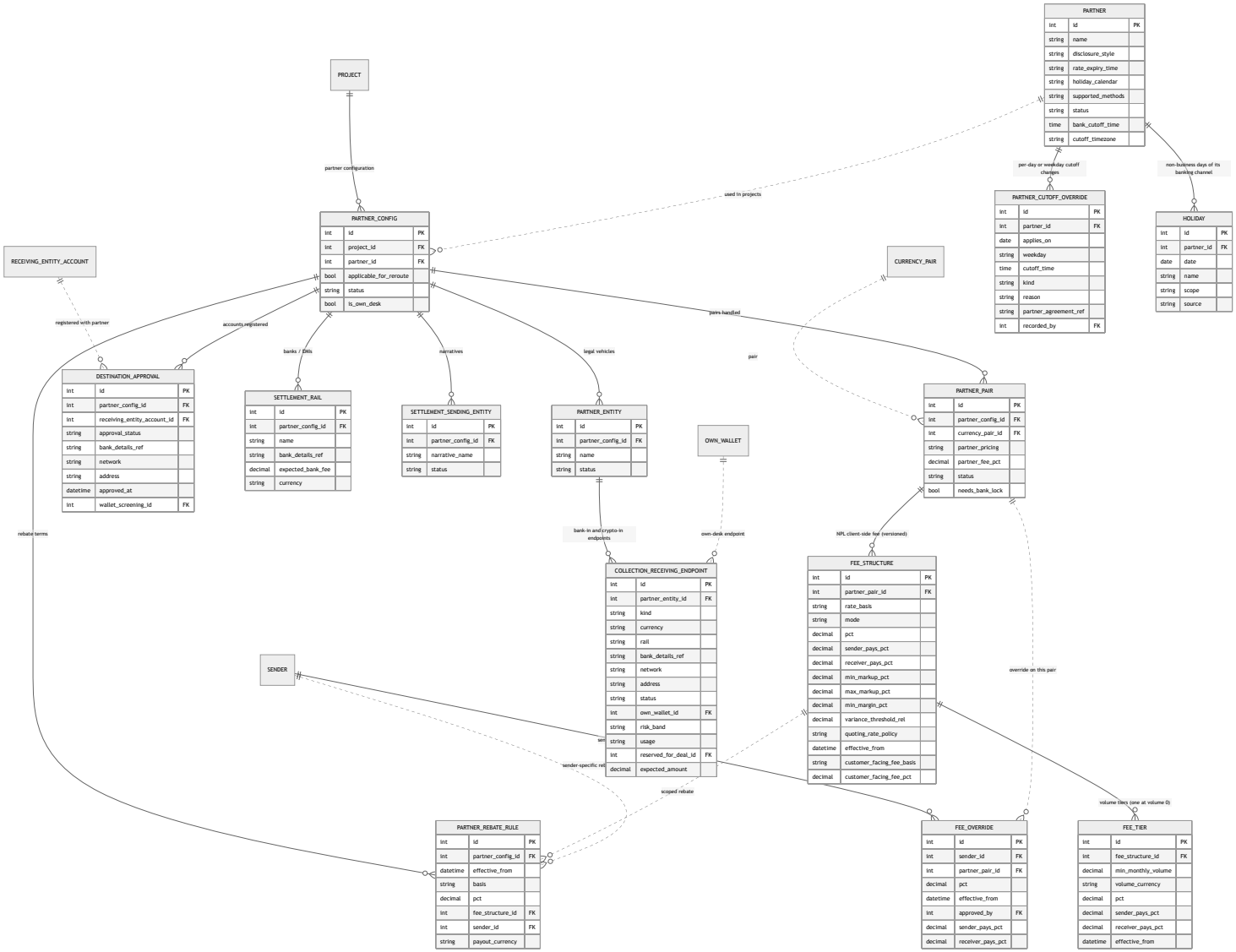


Diagram 1b — Configuration — partner configuration block, fees and tiers

Notes

- A **Client** is the commercial relationship NPL contracts with; a **Project** is one arrangement with it (D38) and owns its configuration. Counterparty receivers sit in **receiver groups**: every project has one default group and every counterparty receiver belongs to exactly one group. Entitlement is tracked per group, so payouts within a group are ordinary settlement and only payouts across groups are offsets; a project with a single group never has an offset.
- A **Receiver** has a kind: `counterparty` for a client receiver; `sender_return` stands for a sender so refunds use the ordinary payout path and the partner's destination approval; `partner_transit` stands for a partner's collection endpoint so the hop of a two-leg route is an ordinary payout. `sender_return` and `partner_transit` receivers belong to no group (`group_id` is null). They never appear on forward deals or allow lists — both validate kind = counterparty — and a payout to them is attributed to the group of the leg that pays them, so no entitlement moves between groups on a refund or a hop. Groups exist for entitlement and offset accounting; refund exclusion is a receiver-kind rule, not a group (D13).
- A receiving entity holds one or more **accounts**, each with a currency and rail (EUR via SWIFT and EUR via SEPA are two accounts). An account is a bank account or a wallet (`account_kind`): a bank account carries `bank_details_ref` and a bank rail; a wallet carries `network` and `address` explicitly, mirroring the partner-side `COLLECTION_RECEIVING_ENDPOINT`, so addresses can be checksum-validated, checked against the rail, and found when the same address is registered with two partners. Destination approvals and settlement lines point at the account. No table stores actual bank details: `bank_details_ref` points at the external store.
- Fees are per partner × pair (`FEE_STRUCTURE` under `PARTNER_PAIR`), so choosing the partner is part of pricing. `PARTNER.disclosure_style` records whether the partner's markup is visible; `partner_fee_pct` is known only for fee_on_market partners.
- A partner operates through one or more legal vehicles (`PARTNER_ENTITY`: PT-tour, PT Global Inc); the deal stamps the vehicle at quote so paperwork names the right company. Each vehicle owns the endpoints a sender can pay into (`COLLECTION_RECEIVING_ENDPOINT`): kind = bank with a currency and `bank_details_ref`, or kind = wallet with `network` and `address`. The partner side therefore mirrors the client side — `PARTNER_CONFIG` → `PARTNER_ENTITY` → `COLLECTION_RECEIVING_ENDPOINT` as `RECEIVER` → `RECEIVING_ENTITY` → `RECEIVING_ENTITY_ACCOUNT` (D15). An endpoint also carries a nullable `rail`: a bank endpoint must have it before it becomes active, and a wallet may leave it null but must match `network` when it is set (D18); the endpoint's kind must match the leg's `collection_method` (D19). Cash-in has no endpoint record; the leg carries the city and the collection the token (D30). The collected cash amount is rounded by the pair's `cash_rounding` option — truncate, nearest or round up — to `cash_rounding_unit` (100 / 500 / 1,000 by currency), the same menu as payout rounding (D25).
- Destination approval is per partner: `DESTINATION_APPROVAL` is an account of the project's receiving entity as registered with this partner, carrying the partner's approval status and the payout details the partner holds. Those details follow the account's kind: `bank_details_ref` for a bank account, `network` and `address` for a wallet (D16). `SETTLEMENT_SENDING_ENTITY` is the narrative on an outgoing transfer; `SETTLEMENT_RAIL` is the bank/EMI that executes it; neither holds money. `COLLECTION_SENDING_ENTITY` is provenance only and never a payout destination.
- `OWN_WALLET` has a role: reroute contingency (AQN-LT-Sub at Aquanow), own desk, or a collection bridge on a network a partner lacks, forwarded to the partner as a hop within the wallet's custody limit (D40); `PARTNER_CONFIG.applicable_for_reroute` defines the alternate-partner set. `PARTNER.supported_methods` declares what a partner offers — cash and local deposits have no standing endpoint to read it from (D39). `WALLET_REGISTRY` lists every known address with its owner and role. Under an **own-desk** partner configuration (`is_own_desk`, the NPL-GR game-reseller project) NPL itself is the converting party: its pairs are priced market plus NPL's markup and its collection endpoints are NPL's own wallets (`COLLECTION_RECEIVING_ENDPOINT.own_wallet_id`); the custody view reports every such balance (D23).
- **Fees in v5.3.** A `FEE_OVERRIDE` may carry its own sender / receiver split instead of inheriting the structure's proportion (D20). `FEE_TIER` rows under a structure price by the project's calendar-month volume; a leg that takes the month past a threshold is split across two tiers (D21). `customer_facing_fee_basis` / `customer_facing_fee_pct` on the structure say what the customer's quote prints when it differs from the recorded parts; the printed "source rate" is derived at quote, never typed (D22). `CURRENCY_PAIR.amount_rounding` is a menu — truncate, nearest or round up, to the unit or to `rounding_unit` — and a pair may have the same currency on both sides (D25, D33).
- **Who else earns on a deal.** `MARKUP_SHARE_PARTY` / `MARKUP_SHARE_RULE` record a share NPL pays — or, with basis `share_of_net_markup` and `party_kind` client_party, the customer who provides a sub-account and shares NPL's markup (Raeen): the base is the leg's fee parts less its partner cost, rebates excluded, several parties may hold rules

with different shares, and `settlement_mode` party_retains means the party holds the converted money and pays NPL the remainder monthly (D26). `PARTNER_REBATE_RULE` records the rebate a partner pays NPL, with a basis (percentage of amount or of the partner's fee) and an optional scope by sender or fee structure; the rebate is NPL's alone and is reported as its own stream (D27).

- **Cutoff, screening, single use.** `PARTNER.bank_cutoff_time` (the partner's own clock, beside its rate expiry and holiday calendar), its per-day changes in `PARTNER_CUTOFF_OVERRIDE` (an agreed extension or an early close) and `PARTNER_PAIR.needs_bank_lock` drive same-day or next-day conversion and, on bank rails, same-day or next-day settlement (D29). Cash legs carry their city (`DEAL.cash_location`) and cash rate versions theirs (D30). `WALLET_SCREENING` records the screening of a sending or receiving wallet address, by Operations, and the endpoint's `risk_band` says which decisions may land on it (D31). An endpoint or a receiving-entity account may be `single_use`, reserved for one deal (ad hoc local deposit accounts, D32). `HOLIDAY` rows under the partner are the dates behind every "next business day" (D34); `MESSAGE_TEMPLATE` is the versioned wording of outgoing messages and declares the fields it may print — a quote template prints sender-facing figures only (D36).

2 · Rates domain

Diagram 2. `RATE_SOURCE`, `CURRENCY_PAIR` are defined in Diagram 1a; `PARTNER_PAIR` in 1b; `DEAL` in 3a; `CONVERSION`, `BALANCE_CONVERSION` in 3b.

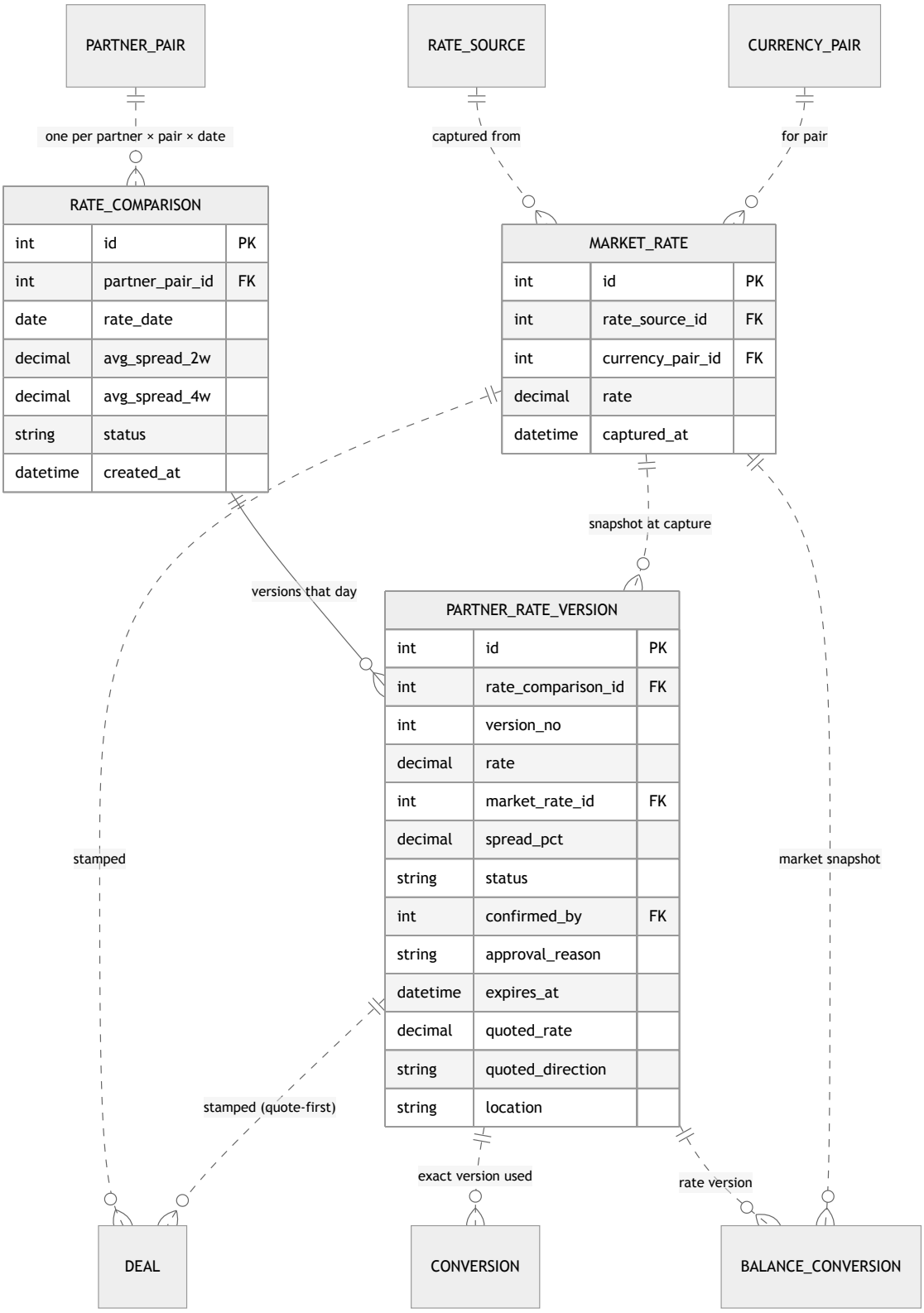


Diagram 2 — Rates

Notes

- The rate comparison for the day is created lazily by the first deal that needs it, one per partner × pair × date, and reused by every deal that day. A partner may give a new rate during the day, so one date can hold several versions. Rolling averages are built from confirmed versions only.
- A version is breached when spread > avg_2w × (1 + variance_threshold_rel); quoting and conversion on that partner × pair freeze until Finance approves with a reason, rejects, or asks for a better rate. Versions are never overwritten; every deal leg, conversion and balance conversion references its exact version.
- Cross-source slippage is a query over `MARKET_RATE` across sources for the same pair and window.
- A version stores the rate **as the partner quoted it** (`quoted_rate`, `quoted_direction`); `rate` is normalised to the pair's direction and derived — 1 ÷ quoted when the directions differ — so Ali's 1.1738 becomes 0.851934 without a hand inversion, and a direction slip is caught by the breach check (D28). Cash pairs are priced per `location`: INR cash in Delhi and in Mumbai are two versions on the same day (D30).

3 · Deal lifecycle domain

3a · Deal group, legs, pricing stamps, invoice, collection

Diagram 3a. `PROJECT`, `SENDER`, `RECEIVER_GROUP`, `RECEIVER`, `COLLECTION_SENDING_ENTITY`, `WALLET_SCREENING`, `MESSAGE_TEMPLATE` are defined in Diagram 1a; `PARTNER_PAIR`, `PARTNER_ENTITY`, `FEE_STRUCTURE`, `FEE_OVERRIDE`, `FEE_TIER`, `COLLECTION_RECEIVING_ENDPOINT` in 1b.

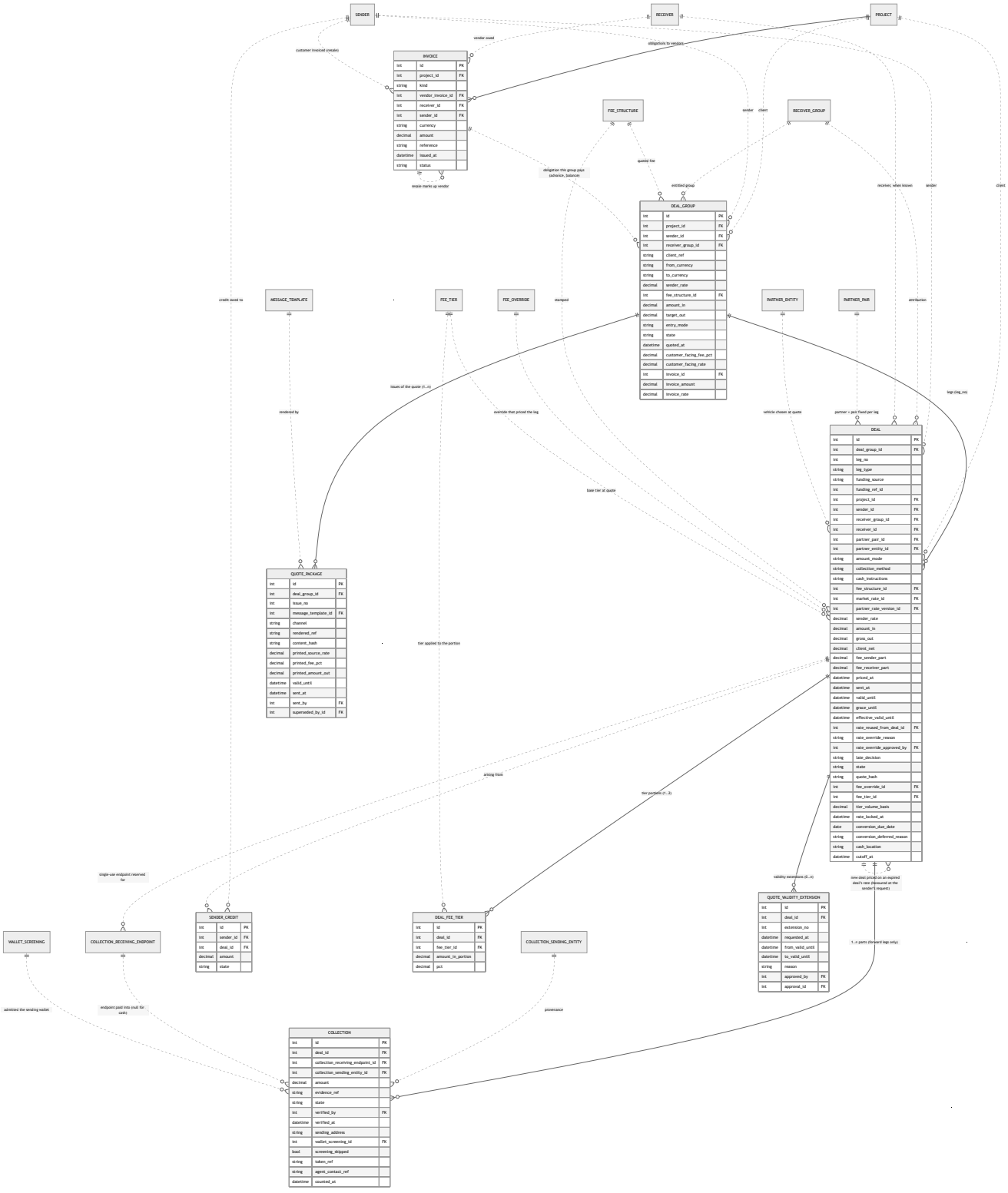


Diagram 3a — Deal lifecycle — deal group, legs, pricing stamps, invoice, collection

3b · Conversion, reroute, earnings receivable, balance conversion, entitlement re-attribution

Diagram 3b. `PROJECT`, `RECEIVER_GROUP`, `OWN_WALLET` are defined in Diagram 1a; `PARTNER_CONFIG`, `PARTNER_PAIR` in 1b; `PARTNER_RATE_VERSION`, `MARKET_RATE` in 2; `DEAL` in 3a.

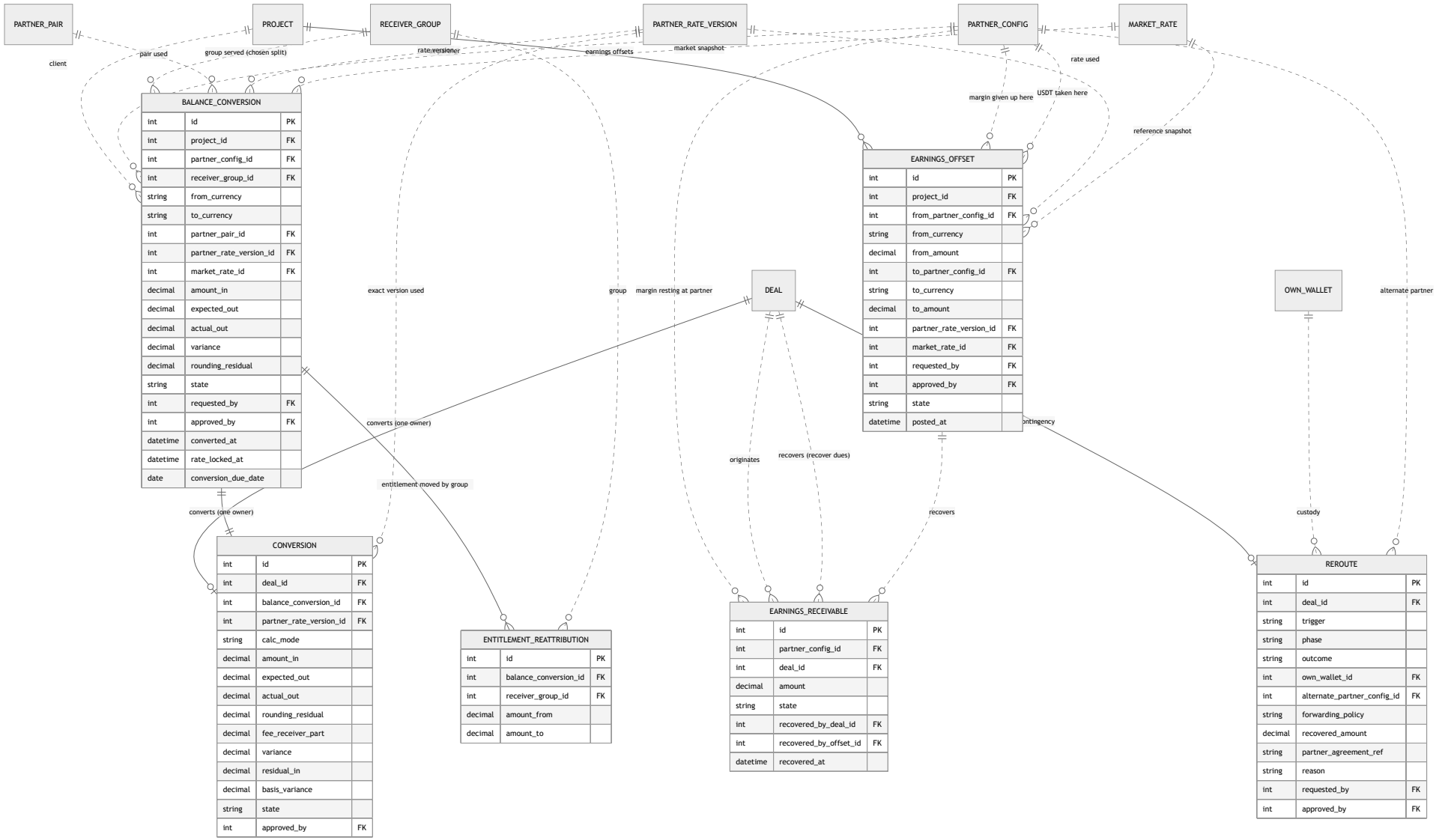


Diagram 3b — Deal lifecycle — conversion, reroute, earnings receivable and offset, balance conversion, entitlement re-attribution

3c · Settlement, return, confirmation, fees, referrals, rebates and loss events

Diagram 3c. `PROJECT`, `RECEIVER`, `MARKUP_SHARE_RULE` are defined in Diagram 1a; `PARTNER_CONFIG`, `SETTLEMENT_SENDING_ENTITY`, `SETTLEMENT_RAIL`, `DESTINATION_APPROVAL`, `PARTNER_REBATE_RULE` in 1b; `MARKET_RATE` in 2; `DEAL`, `COLLECTION`, `INVOICE` in 3a; `BALANCE_CONVERSION` in 3b.

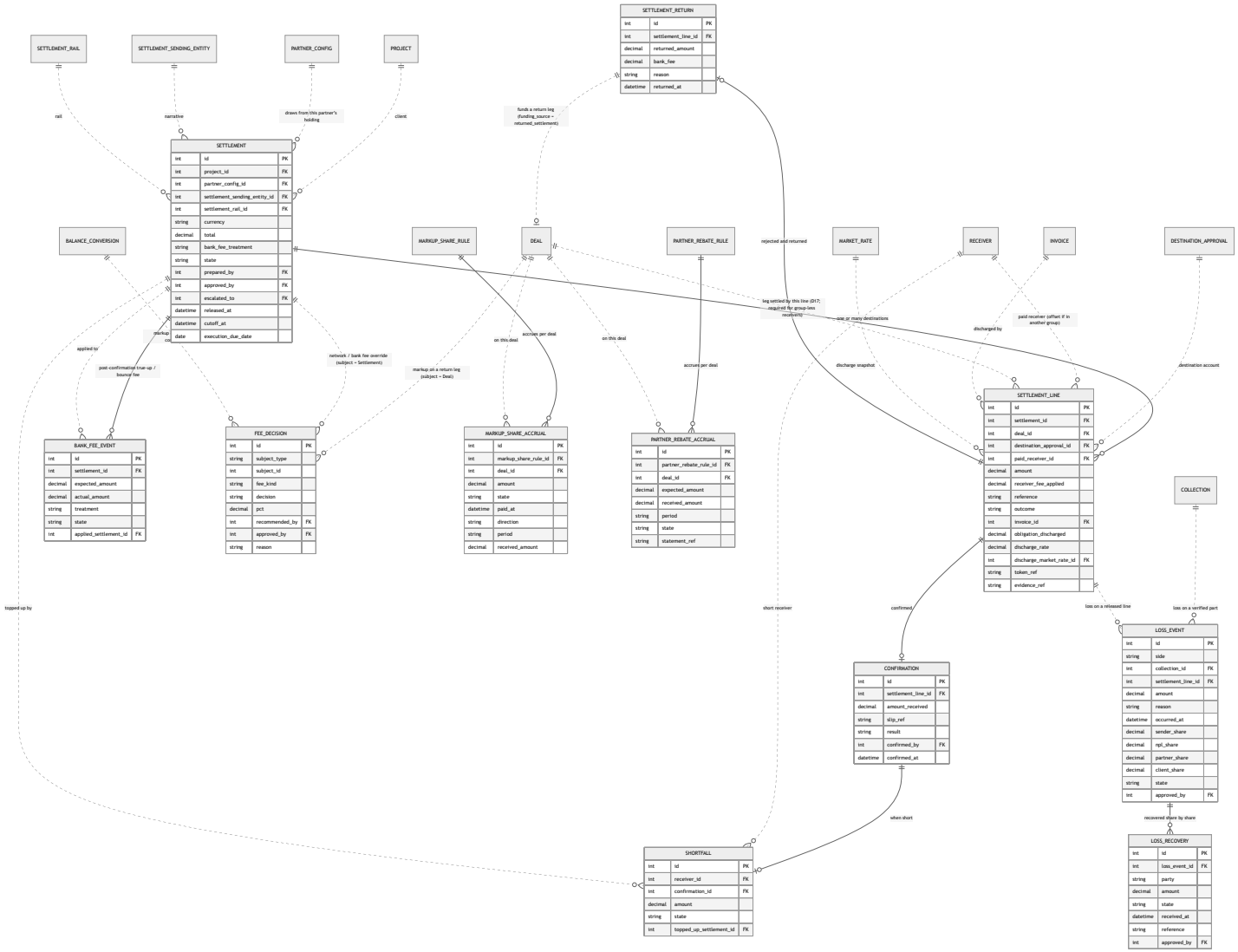


Diagram 3c — Settlement, return, confirmation, fees, shares, rebates, loss events and recoveries

Derived views (never stored)

View	Definition
`PROJECT_BALANCE(project, partner, currency)`	PROJECT_BALANCE(project, partner, currency) = the balance of CL.<PARTNER>.<CCY>.DUE for the partner configuration: Σ debits − Σ credits over postings — conversions in, top-ups, make-goods, offsets and hops in, releases and offsets out — never a sum over deals (D46 amendment: the v5.3 definition summed conversions' actual_out, which includes NPL's margin, and had no term for the v5.7 events). Was CLIENT_BALANCE until v5.4 (D38). Also reports a pending column — the expected_out of legs whose rate is locked with the partner but whose money has not yet been collected or converted; a void removes it; pending is never a posting (D29).
`GROUP_ENTITLEMENT(group, currency)`	GROUP_ENTITLEMENT(group, currency) = Σ credits − Σ debits over CL.PROJECT.<CCY>.PAYABLE postings carrying receiver_group_id = the group: promises raised at collection and conversion, re-raised on a return or a short confirmation, re-denominated by an offset or a balance conversion, reduced by a release or a client loss share. A payout to a group-less receiver carries the paying leg's group (SETTLEMENT_LINE.deal_id, D17). Σ over a project's groups = the PAYABLE balance = Σ client assets, per currency (invariant 7). D46 amendment: over postings, no longer over legs and lines.
`OFFSET(group, currency)`	OFFSET(group, currency) = GROUP_ENTITLEMENT; positive = owed, negative = holding another group's flow. Arises only when a counterparty receiver is paid from a leg attributed to another group (v5.0); zero by construction in a single-group project. Carries Management approval and tighter aging.
`CUSTODY(own_wallet)`	CUSTODY(own_wallet) = Σ collections into the own wallet under open reroutes + Σ recovered dues resting there + Σ own-desk client balances (D23) + Σ bridge collections not yet forwarded (D40) − Σ forwarded or paid out; each row with deal, reason, role and age. The standing control on money in NPL's hands.
`INVOICE_BALANCE(invoice)`	INVOICE_BALANCE(invoice): for a vendor invoice, INVOICE.amount − Σ SETTLEMENT_LINE.obligation_discharged over released lines not returned (a payment discharges on release, D46); for a resale invoice, INVOICE.amount − Σ DEAL_GROUP.invoice_amount over the groups that collected it with a verified collection. Σ VENDOR_PAYABLE = Σ vendor-invoice balances; Σ RESALE_RECEIVABLE = Σ resale-invoice balances (v5.3 D24; restated D46 amendment).
`PROJECT_MONTH_VOLUME(project, month, currency)`	PROJECT_MONTH_VOLUME(project, month, currency) = Σ DEAL.amount_in over forward legs of the project collected in the calendar month, in the tier's volume currency. Read at quote to choose the FEE_TIER; stamped on the leg as tier_volume_basis (v5.3, D21).
`CLIENT_POSITION(client, currency)`	CLIENT_POSITION(client, currency) = Σ over the client's projects of PROJECT_BALANCE (per partner), GROUP_ENTITLEMENT, CUSTODY, and the period's earnings (EARN_GROSS) and rebates (EARN_REBATE) from postings — the client-level report that project = client made impossible. Compared with CLIENT.exposure_ceiling when set (v5.4, D38). Its one-currency total is a REPORTING_VALUE, an estimate at the latest market rate (D43).
`WALLET_REGISTRY(network, address)`	WALLET_REGISTRY(network, address) = every wallet address the model knows, with its owner and role: the partners' receiving wallets (COLLECTION_RECEIVING_ENDPOINT kind wallet, with risk_band), NPL's own wallets (OWN_WALLET with role), receivers' wallets (RECEIVING_ENTITY_ACCOUNT kind wallet, with their DESTINATION_APPROVALs) and the sender addresses seen in WALLET_SCREENING with their latest decision. A view, never a second copy of an address: it answers which networks a partner accepts, which wallet is the bridge, and whether an address was seen before under another owner — the FRS 'wallet registry' (v5.4, D40).

View	Definition
`REPORTING_VALUE(subject, currency, as_of)`	REPORTING_VALUE(subject, currency, as_of) = any per-currency figure of the model — a PROJECT_BALANCE, a CLIENT_POSITION line, a month's collection volume (PROJECT_MONTH_VOLUME), earnings and rebates from postings, custody — multiplied by the latest MARKET_RATE of the platform pair for that currency (its PAIR_RATE_SOURCE chain), with the market_rate_id and captured_at it used and a stale flag when the rate is older than reporting_rate_max_age_min. An estimate for display: the figures in each currency stay the facts, the converted total is a view over them at the moment it is read (v5.4, D43). Serves project-level and overall collection volumes, earnings and current balances in one currency.
`RESELLER_MARGIN(resale_invoice)`	RESELLER_MARGIN(resale invoice) = resale INVOICE.amount – the amount of INVOICE.vendor_invoice_id, in the invoice currency, equal to RESELL_REVENUE – RESELL_COST posted for the pair: NPL-GR's game reseller revenue, fixed when the two invoices are posted and untouched by anything the remittance side does (v5.7, D46; paired through vendor_invoice_id, amendment).
`REMITTANCE_MARGIN(deal)`	REMITTANCE_MARGIN(deal) = EARN_GROSS (net of markup shares) – EXP_PARTNER ± VAR_FX_TIMING – EXP_BANKFEE – EXP_NETWORK – EXP_ROUNDING on the deal's postings, in the converted currency: NPL's remittance earnings on one deal, including the forex result of instalments paid at different days' rates (v5.7, D46).
`NPL_PROFIT(scope, period, currency)`	NPL_PROFIT(scope, period, currency) = remittance earnings (EARN_GROSS + EARN_REBATE – EXP_PARTNER – EXP_BANKFEE – EXP_NETWORK – EXP_ROUNDING – EXP_LOSS ± VAR_CONVERSION ± VAR_CUTOFF ± VAR_RATE_HONOUR ± VAR_FX_TIMING) + reseller earnings (RESELL_REVENUE – RESELL_COST), each callable on its own and together, by project, partner, pair or month, per currency; one figure across currencies is a REPORTING_VALUE (v5.7, D46).

Notes

- **Every deal belongs to a deal group.** `DEAL_GROUP` is the client-facing instruction: the pair as the sender sees it, the sender rate, fee, reference and entry mode. A plain single-partner deal is a group of one leg; a route that needs two partners (INR → USDT at Ali, USDT → EUR at Jeton) has two forward legs; a rejected payout is unwound with a return leg.
- **Legs.** `leg_type` forward or return; `funding_source` collection, prior_leg (leg 2 of a route, no collection) or returned_settlement (a return leg). Priced once and partner fixed at quote hold per leg. Attribution is to the receiver group: set at quote on the first forward leg, inherited by leg n of a route from leg n–1 and by a return leg from its parent leg. The receiver is optional until settlement.
- Two entry modes as before: collect-first legs are born in state Collected with priced_at = conversion time; quote-first legs carry sent / valid / grace timestamps and a late decision. An estimate-first mode gives an indicative figure and prices at conversion.
- **Conversion has exactly one owner**: a deal leg or a `BALANCE_CONVERSION`. The receiver-side fee is taken at conversion. Amounts are rounded at conversion by the pair's option (D25); a dropped fraction (`rounding_residual`, signed) stays client money in the balance and pays out when it reaches a unit, an amount added by rounding up is NPL's cost. NPL's margin is captured at conversion and is never again at risk from a return or a forward.
- **Balance conversion** is NPL converting a client's held balance (5K SGD → USDT to fund a 15K USDT payout): Finance-initiated, Management-approved, no sender, receiver or collection, same rate machinery, markup by `FEE_DECISION`. It names the receiver group it serves; `ENTITLEMENT_REATTRIBUTION` moves that group's entitlement to the new currency, and any excess over the group's entitlement is booked as an approved offset.
- **Reroute** is the only way partner or partner entity change after quote. Forwarding is forward_full or forward_and_recover_dues: NPL nets what the alternate partner owes it (`EARNINGS_RECEIVABLE`) against the funds forwarded, within the outstanding receivable, with the partner's agreement recorded. Post-collection outcome is via the own wallet or a refund to the sender.
- One settlement model: `SETTLEMENT` drains one partner's holding into one or many `SETTLEMENT_LINE`s, rounded as the pair says. Under settlement_policy to_zero the balance after release is less than one unit; under hold_allowed the balance may rest at the partner within the project's exposure cap.

- A line paying a counterparty receiver whose group differs from the paying leg's attributed group is an offset, created by the same action, with Management approval and tighter aging. A line paying a sender_return or partner_transit receiver is never an offset: those receivers have no group, and the line debits the group of the leg it names in `deal_id` (D13, D17).
- **Rejected payout.** When the receiving or intermediary bank rejects a transfer, `SETTLEMENT_RETURN` records the returned amount, bank fee and reason; the line's outcome becomes returned, a reversing ledger transaction puts the money back into the client's balance and the paying leg's group entitlement, and `BANK_FEE_EVENT` books the bounce fee. The money then stays in balance for later payouts, or funds a return leg — attributed to the same group — that converts back and pays the sender's return receiver.
- **Fee decisions.** `FEE_DECISION` is a one-off, approved decision on a single subject: markup applied or waived on a return leg or balance conversion, a network-fee override on a refund, a bank-fee treatment override on one settlement. Standing rules stay in `FEE_STRUCTURE` and `FEE_OVERRIDE`.
- `CONFIRMATION` is full or short; a short confirmation opens a `SHORTFALL` topped up by a later settlement or an explicit separate payout — cash found short at hand-over included. Rounding differences no longer land here. `MARKUP_SHARE_ACCRUAL` and `PARTNER_REBATE_ACCRUAL` accrue per deal under their rules; a party_retains markup-share accrual is a receivable from the party, reconciled monthly like a rebate (D26); rebates are reconciled monthly against the partner's statement, which covers NPLify deals only (D27).
- **Pricing stamps (v5.3).** A leg stamps the override and the base tier that priced it; a leg that crosses a tier threshold holds two `DEAL_FEE_TIER` portions and a blended fee (D20, D21). The deal group stamps the customer-facing fee and rate the quote printed (D22). On a pair that needs a bank lock the leg records `rate_locked_at` and `conversion_due_date`: locked before the partner's cutoff it converts the same day, otherwise the next business day at that day's version; the sender's figures are write-once, NPL absorbs the difference as a cutoff-timing variance, and a conversion whose next-day rate breaches the margin is deferred with a reason. The client balance view shows the locked leg's converted value as pending from the lock; a void removes it (D29). `QUOTE_PACKAGE` is the record of what was sent for a deal group — template version, channel, printed figures, `sent_at`, hash — one row per issue; a quote-first leg is Sent only when a package has sent_at, re-issues are new rows, a new price is a new deal group (D35).
- **Invoices.** `INVOICE` is an obligation in its own currency — a vendor's EUR invoice a reseller customer pays in THB or USDT, in parts, over days. A deal group names the invoice it pays (advance, then balance); each payout line records how much of the obligation it discharged and the market snapshot used; `INVOICE_BALANCE` is the open remainder (D24).
- **Cash and crypto parts.** A cash part carries the token (the local-currency bill whose serial identifies the agent), the agent's contact and the partner's count; the leg carries the city, which also selects the rate version (D30). A crypto part carries the sender's declared address and the screening that admitted it, unless the project skips up-front screening (D31).
- **Loss events.** `LOSS_EVENT` records money lost after it was verified or released — a frozen local account, cash short that cannot be recovered, a wrong-chain transfer — on either side, by any method, for a partial or full amount, with the loss split across sender, NPL, partner and client; Management approves the split, one reversing transaction posts it, and a collection-side event moves the part to reversed (D32).

4 · Ledger & controls domain

Diagram 4. `OWN_WALLET`, `RECEIVER`, `SENDER`, `MARKUP_SHARE_PARTY` are defined in Diagram 1a; `PARTNER_CONFIG` in 1b; `COLLECTION`, `INVOICE` in 3a; `CONVERSION`, `BALANCE_CONVERSION`, `REROUTE`, `EARNINGS_OFFSET` in 3b; `SETTLEMENT`, `SETTLEMENT_RETURN`, `CONFIRMATION`, `BANK_FEE_EVENT`, `MARKUP_SHARE_ACCRUAL`, `PARTNER_REBATE_ACCRUAL`, `LOSS_EVENT`, `LOSS_RECOVERY` in 3c.

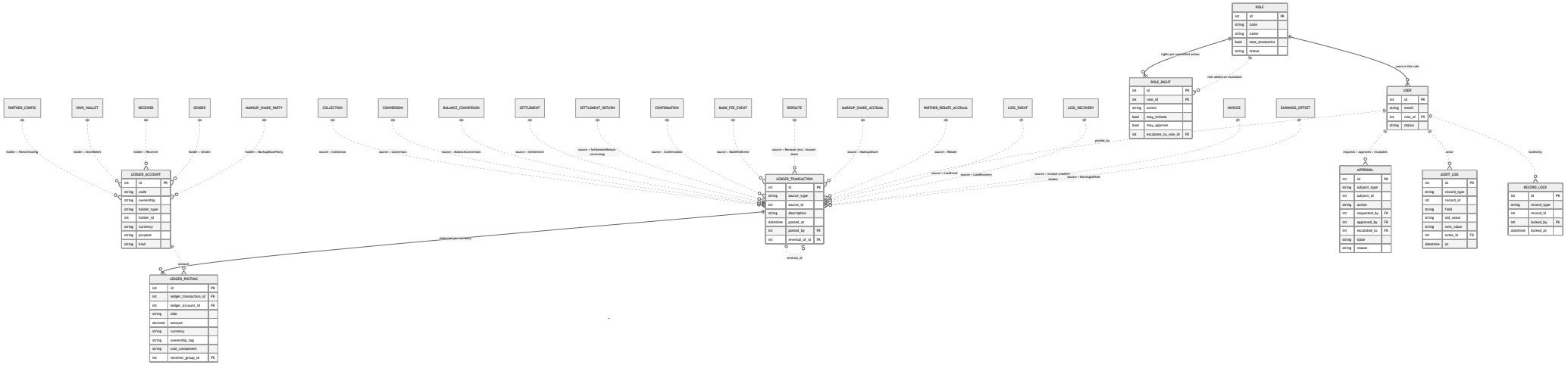


Diagram 4 — Ledger & controls

Notes

- Client due balances are held as one account per project × partner × currency (CL.<PARTNER>.<CCY>.DUE); group entitlement is a view, not an account. `holder_type` never includes a settlement rail or narrative entity.
- **Ownership flips client → company at one event only: margin recognition at conversion.** NPL never advances money to a receiver, and withholding on a reroute is not a separate event because the margin is always captured by whichever partner converts.
- Recover dues on a reroute posts own-wallet cash against the earnings receivable at the partner: company money on both sides, no income.
- Transaction sources: Collection, Conversion, BalanceConversion, Settlement, SettlementReturn, Confirmation, BankFeeEvent, Reroute, MarkupShare, Rebate, LossEvent, Adjustment. Account purposes include PAYABLE, SHARE_PAYABLE, SHARE_RECEIVABLE, REBATE_RECEIVABLE, EARN_GROSS, EARN_REBATE, EXP_ROUNDING and EXP_LOSS; ADVANCE does not exist. Cost components include share, rebate, rounding, fx_timing, cutoff_timing and loss.
- **Three streams NPL never mixes (v5.3).** Earnings on deals post to EARN_GROSS at conversion; a partner's rebate posts to EARN_REBATE and is never in the base a client party shares (D26, D27); NPL's remainder from a party that holds the converted funds sits in SHARE_RECEIVABLE until the party pays it. The cost of rounding up is EXP_ROUNDING, NPL's share of a loss event is EXP_LOSS; the client's and sender's shares of a loss reduce the client balance, the partner's share is a receivable from the partner (D25, D32). Own-desk deals use the ordinary partner-coded accounts with the own-desk configuration as holder and EXP_PARTNER zero by construction (D23).
- One system-settings area (D43, D44): `SYSTEM_SETTING` holds NPL-wide, Management-approved keys — the reporting currency, its staleness limit, the operating time zone; `ROLE` holds the user roles and `ROLE_RIGHT` what each may initiate and approve per controlled action, with the role added on escalation; platform-scope `CURRENCY_PAIR`s carry the conversion rates through the ordinary rate-source chain. `REPORTING_VALUE` converts any per-currency figure into the reporting currency at the latest market rate of its platform pair, as a display-only estimate that carries the rate it used (D43). `APPROVAL`, `AUDIT_LOG` and `RECORD_LOCK` are modelled once and apply to all master data. Approver ≠ initiator on every approval; Operations never sees derived economics. Approval actions include refund_to_third_party, cross_group_payout, balance_conversion, recover_dues, enable_own_desk, approve_loss_split and override_screening_reject. cross_group_payout never fires on a line paying a sender_return or partner_transit receiver. Wallet screening itself is an Operations action, not an approval (D31).

5 · Invariants (enforced at write time)

1. Postings in one transaction sum to zero per currency.
2. Ledger tables are append-only; corrections are reversing transactions.
3. Custody in NPL's own wallet is allowed in three cases only: a collection on a deal with an approved REROUTE; a leg whose PARTNER_CONFIG is an own desk (D23, the NPL-GR project); or a collection on a wallet with role collection_bridge, which is forwarded to the leg's partner as a hop (a SETTLEMENT_LINE to the partner_transit receiver) within the wallet's custody_max_hours (D40). CL.OWN.*.HELD is debited by nothing else; the custody aging view reports all three and is the standing control.
4. Settle to the residual: under settlement_policy = to_zero a SETTLEMENT pays the balance rounded by the pair's option and leaves only the rounding residual (dust stays client money; a round-up is NPL's cost — D25); under hold_allowed the balance may rest at the partner within the project's exposure cap (Q5, Q8).
5. Priced once, per leg: a leg's pricing fields are write-once after priced_at; a re-quote is a new leg or a new deal group.
6. Partner fixed at quote, per leg: partner_pair_id / partner_entity_id change only through an approved REROUTE. A collection's endpoint must belong to the leg's partner entity (v5.1, D15). Its kind must match the leg's collection_method — wallet for crypto, bank for bank, no endpoint for cash (v5.2, D19). A COLLECTION may reference only an active endpoint; a bank endpoint is active only with rail set (D18).
7. Entitlement conservation, over postings: Σ GROUP_ENTITLEMENT over a project's groups = the balance of CL.PROJECT.<CCY>.PAYABLE = Σ COLLECTED + Σ HELD + Σ DUE + Σ SHORTFALL + MAKEGOOD − Σ CREDIT, per currency, after every transaction — because every transaction balances per owner and every PAYABLE line carries a receiver_group_id. Return and hop legs preserve it by inheriting the parent leg's group (D13); a payout to a group-less receiver carries the paying leg's group (D17) (restated, D46 amendment).
8. Destination approval precedes collection: a quote-first deal cannot leave Inquiry unless at least one account in the entitled group has a DESTINATION_APPROVAL in state approved with the chosen partner.
9. Approver ≠ initiator on every APPROVAL, and the approver's ROLE has may_approve for the action in ROLE_RIGHT while the initiator's has may_initiate; Operations never receives derived-economics fields (ROLE.sees_economics) (restated v5.4, D44).
10. Every balance shown anywhere is a sum over postings or deals.
11. A collect-first deal requires an existing SENDER_RECEIVER_ALLOW entry for its sender and group (Q1).
12. A return leg pays the sender_return RECEIVER linked to the parent leg's sender, unless a Management-approved exception with a reason names another approved receiver.
13. The failed forward line is reversed before a return leg opens, so entitlement and balance are correct at the return conversion.
14. A CONVERSION belongs to exactly one owner: a deal leg or a balance conversion.
15. A recovery on a reroute cannot exceed the outstanding EARNINGS_RECEIVABLE at that partner; the alternate partner's expected conversion is computed on the net amount forwarded.
16. Ownership moves between client and company at named events only, and every transaction balances per owner as well as per currency. Client → company: margin recognition at conversion; the client side of an EARNINGS_OFFSET at the to-partner; a forex true-up gain on a line discharging an invoice; the client's share of a loss, by partial reversal of the make-good. Company → client: a fee NPL absorbs or covers (bank, network, rounding up), a shortfall top-up, a bounce make-whole, a loss make-good; the company side of an EARNINGS_OFFSET at the from-partner; a forex true-up loss. NPL never advances money to a receiver (Q13) (restated, D46 amendment).
17. Offsets arise only when a counterparty receiver is paid from a leg attributed to a different group, and carry Management approval and tighter aging. A project with one group never has an offset; within a group nothing is tracked beyond an informational tally (v5.0).
18. A RECEIVER has a group_id if and only if kind = counterparty. Forward legs and the sender allow list target counterparty groups and receivers only. A return leg inherits receiver_group_id from its parent leg; leg n of a route inherits it from leg n−1 (v5.0, D13).

19. A SETTLEMENT_LINE paying a group-less receiver (sender_return, partner_transit) carries deal_id; the paying leg's group is a stored fact, never inferred (v5.2, D17).
20. A FEE_OVERRIDE with its own split has both share columns set and they sum to its pct; with neither set the structure's proportion applies. The leg stamps the override that priced it (v5.3, D20).
21. The base tier stamped on a leg is the tier with the highest min_monthly_volume not exceeding the project's calendar-month volume at quote (PROJECT_MONTH_VOLUME); a leg that takes the month past a threshold is split into two DEAL_FEE_TIER portions whose amounts sum to amount_in, and its fee parts are the blended sum (v5.3, D21).
22. Customer-facing figures are derived, never typed: customer_facing_rate = sender_rate ÷ (1 − customer_facing_fee_pct), stamped at quote; the true rates and fee parts are stored beside them and Operations sees the customer-facing pair only (v5.3, D22).
23. Σ obligation_discharged over a vendor invoice's released lines never exceeds its amount, and Σ DEAL_GROUP.invoice_amount over a resale invoice's groups never exceeds its amount. A line that names a vendor invoice names a deal whose group collects the resale invoice that marks it up (vendor_invoice_id), or, outside a reseller project, whose group is on that invoice; the margin line names the resale invoice itself (v5.3 D24; restated D46 amendment).
24. Rounding is the pair's configured option — amount_rounding / rounding_unit on payouts, cash_rounding / cash_rounding_unit on the cash amount collected; the residual is signed — a dropped fraction stays client money in the balance, an amount added by rounding up is NPL's cost posted to EXP_ROUNDING — so every rounding difference has an owner (v5.3, D25).
25. Σ pct of the share_of_net_markup rules active on one leg ≤ 100 %; the share base is the leg's fee parts less its partner cost and never includes a partner rebate. A MARKUP_SHARE_ACCRUAL is created and posted by the conversion it arises on; a party_retains accrual is a receivable from the party, reconciled monthly (v5.3, D26; restated v5.7, D46).
26. Rebate income posts only to EARN_REBATE, never to EARN_GROSS; every rebate accrual names the NPLify deal it arises on, is created and posted by that deal's conversion as a percentage of the amount in the rule's payout currency, and the partner statement is reconciled against those deals alone (v5.3, D27; restated v5.7, D46 and amendment).
27. A partner rate version stores the rate as quoted and its direction; the normalised rate is derived (1 ÷ quoted when the directions differ, at the pair's precision) and is subject to the breach check like any other, so a direction slip freezes the pair instead of pricing a deal (v5.3, D28).
28. The partner's cutoff binds two events on pairs that rely on the banking channel. The effective cutoff for a day is the one-day PARTNER_CUTOFF_OVERRIDE if any, else the weekday rule, else PARTNER.bank_cutoff_time, and is stamped (cutoff_at) on the leg at rate lock and on the settlement at approval. A leg's conversion_due_date is the lock date when rate_locked_at precedes cutoff_at and the next business day otherwise, and Converting may not start before it; a bank-rail settlement's execution_due_date follows the same rule from its approval time. The sender's figures are write-once (invariant 5); the difference between the stamped partner rate and the rate actually used posts to VAR_CUTOFF with component cutoff_timing — NPL absorbs it (v5.3, D29, amended 9 October).
29. A rate version is unique per partner × pair × day × location; a cash leg carries its cash_location and prices on a version of that location; a cash part is verified only after the partner's count (counted_at) (v5.3, D30).
30. A crypto collection names its sending address and, unless the project skips up-front screening, the screening that admitted it; a screening decided accept_alternate may land only on an endpoint whose risk_band is elevated, and no collection exists for a decision of reject unless Finance overrides it (v5.3, D31).
31. A single_use endpoint or account is referenced by at most one collection or payout line, for the deal it was reserved for, and retires when that reference is verified or confirmed. A LOSS_EVENT's shares sum to its amount, which never exceeds the amount verified or released; a collection-side event moves the part to reversed and the client balance never goes below zero (v5.3, D32).
32. A pair whose two currencies are the same is converted at rate 1 like any other pair, so the sender and receiver fee parts are captured at conversion and the balance arithmetic is unchanged (v5.3, D33).
33. Business days are computed, never assumed: 'next business day' for a conversion due date or a settlement execution date skips weekends in the partner's cutoff_timezone and the partner's HOLIDAY rows of the matching scope; a reroute with trigger holiday names a day that is a HOLIDAY of scope partner (v5.3, D34).

34. A quote-first leg leaves Quoted for Sent only when a QUOTE_PACKAGE of its group has sent_at; the package's content_hash is the leg's quote_hash and its printed figures equal the group's customer-facing figures (derived, never typed); freshness runs from the leg's market snapshot (MARKET_RATE.captured_at) to sent_at within PROJECT.quote_freshness_min, validity from sent_at. Packages are append-only: a re-issue supersedes, a new price is a new deal group. A quote template prints sender-facing fields only (v5.3, D35, D36).
35. Client-level figures are sums over the client's projects, never stored (CLIENT_POSITION); when CLIENT.exposure_ceiling is set, Σ PROJECT_BALANCE across the client's projects may not exceed it — a settlement or a held balance that would breach it needs Management approval, as a project's own ceiling does (v5.4, D38).
36. A partner offers only what its record says: a leg's collection_method, and the method of any settlement through the partner, must be in PARTNER.supported_methods; a wallet endpoint's network must already exist as an endpoint of that partner's vehicle before a collection may name it. Adding a method or a network is a Management-approved configuration change (v5.4, D39).
37. A quote's validity may be extended any number of times before the validity in force ends, each time by an APPROVAL (extend_quote_validity) recorded as a QUOTE_VALIDITY_EXTENSION that moves effective_valid_until and re-issues the quote package at the same price; Finance approves, Management is added when the new validity passes the partner's cutoff or the rate version's expiry. The pricing fields stay write-once (invariant 5). After the validity in force ends, the paths are the grace decision and a new deal group (v5.5, D45; was once-only within grace, D42).
38. Converted figures are display-only estimates: a REPORTING_VALUE is computed at read time from the latest MARKET_RATE of the platform CURRENCY_PAIR (priced through its PAIR_RATE_SOURCE chain), carries the rate and its time, is flagged when the rate is older than reporting_rate_max_age_min, and is never posted, stored, quoted or settled. The only controls that read it are ceilings stated in the reporting currency (CLIENT.exposure_ceiling); every other limit and every posting stays in its own currency (v5.4, D43).
39. An expired quote is never revived: the sender gets a new deal group. At the sender's request that new leg may be priced on the earlier leg's market rate and partner rate version (rate_reused_from_deal_id) by an approved override (honour_expired_rate, Finance), the automated rate set aside and the grace drift limit not applied; the leg stamps the reused rates as its own, and the difference against the partner's actual rate at conversion posts to VAR_RATE_HONOUR — NPL's cost, chosen (v5.5, D45).
40. A settlement discharges CL.PROJECT.<CCY>.PAYABLE when the partner releases the money (released_at); a SETTLEMENT_RETURN re-raises it for what came back and a short CONFIRMATION for the part not delivered; a full confirmation posts nothing; a hop (partner_transit) posts only when the far side credits and never touches the promise. Every released line ends confirmed, short or returned, and 'sent, not yet confirmed' is read from the line, never from an account (v5.7, D46).
41. CO.<PARTNER>.<CCY>.POOL may be below zero only between a fee NPL absorbed in a currency where it held nothing at that partner and the next conversion there, which then includes NPL's own USDT converted alongside the client's (a Conversion with a company part through FX_RESULT); Finance squares every negative pool within the month (v5.7, D46).
42. Fee parts are percentages of the principal and never compound. Fixed-in (amount_mode fixed_in): principal = amount_in × basis rate; fee_sender_part = principal × sender_pays_pct; fee_receiver_part = principal × receiver_pays_pct; client_net = principal × (1 – sender_pays_pct – receiver_pays_pct). Target-out (amount_mode target_out): the fee is a markup on the target — amount_in = target_out × (1 + sender_pays_pct) ÷ basis rate, then cash-rounded if collected in cash; fee_receiver_part = target_out × receiver_pays_pct; client_net = target_out × (1 – receiver_pays_pct) + any cash-rounding surplus; the sender rate printed is basis rate ÷ (1 + sender_pays_pct) (v5.7, D46; target-out added by the amendment — NPL's quotes use × (1 + fee), not ÷ (1 – fee)).
43. An EARNINGS_OFFSET is neutral for the client: from_amount at the from-partner becomes the client's and to_amount = from_amount ÷ the to-partner's rate of the day becomes NPL's, with the client's promises re-denominated by the same amounts; to_amount never exceeds the client's unconverted balance at the to-partner, from_amount never exceeds NPL's pool at the from-partner, and the receivables it recovers are marked recovered_by_offset_id (v5.7, D46).
44. In NPL-GR the vendor and resale invoices are posted once each, in the invoice currency (RESELL_COST / VENDOR_PAYABLE; RESALE_RECEIVABLE / RESELL_REVENUE), and no later transaction touches RESELL_REVENUE or RESELL_COST: the reseller revenue is intact by construction. Each group's collection

clears its invoice_amount of RESALE_RECEIVABLE into REMIT_CLAIM; each vendor line clears VENDOR_PAYABLE and REMIT_CLAIM by its obligation_discharged on release; the margin line clears the rest of REMIT_CLAIM; a cash-rounding surplus is NPL-GR's revenue in the currency received, never a forex result; the forex true-up of each instalment posts to VAR_FX_TIMING on the remittance side (v5.7, D46 and amendment).

6 · What changed from v4.0

Draft v5.7 (11 October 2026) is v5.6 plus D46, the ledger design agreed with NPL on 10–11 October: two new tables (`EARNINGS_OFFSET`, `LOSS_RECOVERY`), three new views (`RESELLER_MARGIN`, `REMITTANCE_MARGIN`, `NPL_PROFIT`), the ledger vocabulary of the Ledger Posting Design v5.7, and invariants 40–44. Draft v5.6 (9 October 2026) was v5.5 with the examples anonymised; v5.5 closed the 8–9 October work: v5.4 added the CLIENT parent (D38) and, through the evening, D39–D44. v5.3 applied the thirteen fee-practice gaps found between NPL's fee table, NPL's workbook answers and the Project Context / Payment methods tabs and ERD v5.2 — written up as proposals A–M in "Model Gaps & Proposals v1.0" and confirmed by NPL on 8–9 October — plus the same-currency rule, as decisions D20–D33. Fourteen tables are added (`CLIENT`, `SYSTEM_SETTING`, `ROLE`, `ROLE_RIGHT`, `QUOTE_VALIDITY_EXTENSION`, `FEE_TIER`, `DEAL_FEE_TIER`, `INVOICE`, `WALLET_SCREENING`, `LOSS_EVENT`, `PARTNER_CUTOFF_OVERRIDE`, `HOLIDAY`, `QUOTE_PACKAGE`, `MESSAGE_TEMPLATE`) and five views (`INVOICE_BALANCE`, `PROJECT_MONTH_VOLUME`, `CLIENT_POSITION`, `WALLET_REGISTRY`, `REPORTING_VALUE`): 70 tables and 12 views. Earlier: v5.0 / v5.1 (7 October) removed the return group (D13), gave accounts explicit wallet fields (D14) and restructured the partner's collection side (D15); v5.2 (8 October 2026) added D16–D19.

v5.3 to v5.7 — the fee-practice decisions, the FRS-term additions, the naming alignment, the client parent, NPL's operational answers and the ledger design (11 October 2026)

D46 — The ledger design: variance by cause, loss first, accruals at conversion, the reseller books, earnings kept unconverted and offset, sent is discharged

Decision. Eight parts, from the Ledger Posting Design v5.7 and NPL's answers of 10–11 October. (1) `LEDGER_ACCOUNT.purpose` VARIANCE becomes VAR_CONVERSION, VAR_CUTOFF, VAR_RATE_HONOUR and VAR_FX_TIMING. (2) A `LOSS_EVENT` is booked to NPL in full and the client made whole in one transaction (EXP_LOSS; MAKEGOOD / LOSS_PAYABLE when it cannot be funded at once); each share is then a `LOSS_RECOVERY` posting; a partial loss posts the lost part only. (3) Markup-share and rebate accruals are created and posted by the conversion; `PARTNER_REBATE_RULE.payout_currency`. (4) NPL-GR keeps two invoices per deal in EUR — `INVOICE.kind` vendor / resale, `sender_id` — posted once as cost and revenue (source Invoice, component resale), so the reseller revenue is intact by construction (`RESELLER_MARGIN`); the remittance deal that collects the resale invoice posts like any client project with NPL-GR as the client, the THB collected as cash and rounded, and each instalment's forex result is a true-up to VAR_FX_TIMING (`REMITTANCE_MARGIN`); `NPL_PROFIT` = remittance + reseller earnings, each callable alone. (5) Earnings may be kept in the unconverted balance (`CONVERSION.residual_in`) and offset between partners (`EARNINGS_OFFSET` at the to-partner's rate of the day, recovering `EARNINGS_RECEIVABLE` rows; Finance requests, Management approves), recorded for NPL in FX_RESULT. (6) FX_RESULT, a profit-and-loss account per currency, for any exchange of NPL's own money. (7) The promise is discharged on release: INTRANSIT retired, a return or a short confirmation re-raises PAYABLE, a full confirmation posts nothing, a hop posts on credit; `CURRENCY_PAIR.amount_rounding` gains exact (bank transfers carry cents, unit rounding is for cash); a pool may be negative only until the next conversion squares it with NPL's own USDT. (8) Fee parts are percentages of the principal at the basis rate, never compounded; network fees in USDT, absent when none applies. Invariants 16, 25, 26, 28, 39 restated; 40–44 new.

Rejected alternative. A single VARIANCE account; the reseller books as invoice records with views only; the market rate for an offset; INTRANSIT as a client asset; recognising the vendor invoice as cost when discharged rather than when received.

Why. NPL (10–11 October): the ledger must show where NPL's margin physically is (it is taken out of unconverted collections and offset between partners), prove the reseller revenue untouched whatever the remittance side does, answer 'NPL's profit' with both businesses, and treat sent money as out of NPL's hands. Fee facts folded in: Evo USDT → EUR sender 1 % or 1.4 % with receiver 0.6 %, EUR → EUR 2.8 % split 50/50, Jeton 1.25 %, Ali assumed 1 % inside its rate, NPL-GR THB collected as cash.

D20 (A) — A per-sender override carries its own split

Decision. `FEE_OVERRIDE` gains `sender_pays_pct` and `receiver_pays_pct`; both blank means the structure's proportion, both set means they sum to `pct`. `DEAL` stamps `fee_override_id`.

Rejected alternative. Keeping the override as a total only, with the structure's proportion.

Why. Evo's selected senders pay 1.0 % while Evo still pays its 0.6 %; a proportional override would charge 1.12 % and 0.48 %. NPL confirmed the receiver part is fixed for every sender.

D21 (B) — Fees by monthly volume

Decision. `FEE_TIER` under a fee structure (one tier at volume 0); the tier is chosen at quote from the project's calendar-month volume (`PROJECT_MONTH_VOLUME`) and stamped with the volume that justified it. A leg that takes the month past a threshold is split: `DEAL_FEE_TIER` holds the portion below the threshold at the lower tier and the portion above at the next, and the leg's fee parts are the blended sum.

Rejected alternative. A rolling 30-day window; switching the whole leg to the new tier; switching from the next leg only.

Why. GDC pays 2.5 % below USD 500,000 a month and 2.25 % above, switched by hand today. NPL: calendar month; the crossing transaction itself attributes the part above the threshold to the lower tier.

D22 (C) — What the quote prints

Decision. `FEE_STRUCTURE.customer_facing_fee_basis` (actual / sender_pays_on_agent_rate / fixed) and `customer_facing_fee_pct`; `DEAL_GROUP` stamps `customer_facing_fee_pct` and `customer_facing_rate` = sender_rate ÷ (1 – customer-facing fee) so a re-printed quote matches. Customer-facing figures are derived, never typed; Operations sees the customer-facing pair, Finance both.

Rejected alternative. Leaving the recomputation of the 'market rate' to Operations by hand.

Why. BF quotes via Ali show 'fee 1 %' while the recorded parts add to 1.4 % (Ali's 1 % inside its rate plus NPL's 0.4 %). NPL: a BF-only rule, to be cleared when the project moves to partner rate plus markup. The Raeen sub-account quote uses the same recomputation.

D23 (D) — NPL-GR as an own-desk project

Decision. `PARTNER_CONFIG.is_own_desk`; an own wallet is a `COLLECTION_RECEIVING_ENDPOINT` under it (`own_wallet_id`); own-desk pairs are priced market plus NPL's markup with generated rate versions; invariant 3 restated: client money may rest in NPL's wallet under an approved reroute or in an own-desk project, and the custody view reports both. Enabling an own desk is a Management-approved configuration change (`enable_own_desk`).

Rejected alternative. Keeping the no-custody principle intact and running the game-reseller flows outside NPLify.

Why. The reseller products (Bala, Fair, 66 Group, TripleSeven, Alpha Plus, 27 Group via THB cash) collect into NPL's own wallet and pay vendors from it on later days; there is no outside partner for the conversion. NPL's framing: NPL-GR, a business unit of NPL, is the client. Management confirmed running it inside the system.

D24 (E) — Invoices paid in parts

Decision. `INVOICE` owned by the project, naming the vendor; `DEAL_GROUP.invoice_id`; `SETTLEMENT_LINE` gains `invoice_id`, `obligation_discharged`, `discharge_rate` and the market snapshot used; `INVOICE_BALANCE` = amount – Σ discharged over confirmed lines. The EUR difference between what was collected and what was discharged posts to variance with component fx_timing.

Rejected alternative. Tracking the obligation in a free-text client reference.

Why. A vendor's EUR invoice is settled in USDT in instalments at different days' rates (EUR/USDT 0.8708 at collection, 0.87527 at the last payment); the open balance changes with every instalment. NPL: wanted; vendors are paid on different days and the forex difference is accepted within the 2 % markup.

D25 (F) — A rounding menu per pair

Decision. `CURRENCY_PAIR.amount_rounding` grows from one value to truncate_unit / nearest_unit / round_up_unit / nearest_n / round_up_n with `rounding_unit`; `CONVERSION.rounding_residual` is signed — a dropped fraction stays client money, an amount added by rounding up is NPL's cost in `EXP_ROUNDING`. Cash collections use the same menu on the collected amount (`cash_rounding` with `cash_rounding_unit`).

Rejected alternative. Truncation only, with the other practices left outside the system.

Why. Live quotes round THB and INR to the nearest 1,000 and round a reseller customer's USDT up. NPL: round up to the nearest 1,000 for Asian currencies, nearest unit for USD, SGD, HKD; round-up on USDT is deliberate.

D26 (G) — Markup shared with a client party

Decision. `MARKUP_SHARE_RULE` basis `share_of_net_markup`: the base is the leg's fee parts less its partner cost, on amount × market, rebates never included; `party_kind` markup-share party / client_party; `settlement_mode` npl_pays / party_retains; optional scope by fee structure; several rules per project with Σ shares ≤ 100 %. `MARKUP_SHARE_ACCRUAL` gains direction, period and received_amount; a party_retains accrual is a receivable (`SHARE_RECEIVABLE`) reconciled monthly.

Rejected alternative. A fixed percentage per party, reconfigured when the route changes; NPL always paying the share out.

Why. Raeen's downlines fund sub-accounts at 4 %; whatever is left after the partner's fee is split equally — 1.5 % each via Ali, 1.6 % each via Aquanow — Raeen keeps the converted EUR in its own Aquanow account and pays NPL monthly; direct downlines split 0.2 % 75 / 25. NPL: future deals may have several parties with different shares; the partner's rebate is NPL's alone and not visible to the party.

D27 (H) — Rebates as their own stream

Decision. Rebate income posts to `EARN_REBATE`, never `EARN_GROSS`. `PARTNER_REBATE_RULE` gains `basis` (pct_of_amount / pct_of_partner_fee) and `pct`, retires the old `pct_of_partner_fee` column (amended 9 October), and gains an optional scope by sender or fee structure. `PARTNER_REBATE_ACCRUAL` keeps `deal_id` required and gains `statement_ref`.

Rejected alternative. Accruals without a deal for clients NPL referred but does not service.

Why. NPL: the partner statement that NPLify reconciles covers only deals NPLify handled — customers working with Jeton or Aquanow directly are out of scope. Jeton pays 0.4 % of the amount on a 1 % or 1.25 % fee and 0.3 % on Novi's 0.9 %, so the rebate is not a fixed share of the fee. Management reads earnings and rebates as two numbers.

D28 (I) — Partner rates as quoted

Decision. `PARTNER_RATE_VERSION` gains `quoted_rate` and `quoted_direction`; `rate` is derived (1 ÷ quoted when the directions differ, at the pair's precision) and goes through the breach check like any rate.

Rejected alternative. A single normalised rate typed by Operations.

Why. Ali sometimes states USDT → EUR as 1.1738 (EUR-per-USDT the other way round); the hand inversion is a 38 % error waiting to happen, and the audit trail should show the number the partner sent.

D29 (J) — Bank cutoff, rate lock, next-day conversion

Decision. `PARTNER.bank_cutoff_time` / `cutoff_timezone` (amended 9 October: on PARTNER, not PARTNER_CONFIG — the cutoff is the partner's, whichever project uses it), with `PARTNER_CUTOFF_OVERRIDE` for a one-day extension the partner agreed (30–60 minutes, urgent requests) or an early close (Fridays, pre-holiday); `PARTNER_PAIR.needs_bank_lock` for the pairs that rely on the banking channel; the effective cutoff is stamped as `cutoff_at` on the leg at rate lock and on the settlement at approval, and `SETTLEMENT.execution_due_date` applies the same rule to the outgoing bank transfer — the second event the cutoff governs; `DEAL` and `BALANCE_CONVERSION` carry `rate_locked_at` and `conversion_due_date` (the lock date before the cutoff, else the next business day; Converting may not start earlier) and a `conversion_deferred_reason`. The client balance view shows a locked leg's converted value as pending from the lock; a void removes it. The difference between the stamped partner rate and the one used posts to variance with component cutoff_timing.

Rejected alternative. Treating the cutoff as part of quote validity; re-quoting the sender the next day.

Why. Evo via Ali: the rate is compared in the morning; locked before 14:30 GMT+8 it converts the same day, otherwise the money is collected on the morning's figures and converted the next business day. AK's USD and SGD balances are banking-ready and wait too. NPL: NPL absorbs the difference; if Ali's next-day rate breaches the margin, ask for a better rate or wait a day; today's balances should show the pending converted value.

D30 (K) — Cash by token, rates per city

Decision. `DEAL.cash_location`; `PARTNER_RATE_VERSION.location` with uniqueness per partner × pair × day × location; `COLLECTION` gains `token_ref`, `agent_contact_ref` and `counted_at`, with cash states token_issued → handed_over → counted → verified; `SETTLEMENT_LINE.token_ref` for cash payouts.

Rejected alternative. Free-text cash instructions on the deal and one rate per currency; making the city a side of the currency pair (CASH_LOCATION) — proposed and withdrawn on 9 October at NPL's request.

Why. A cash collection or settlement is arranged by token — a local-currency bill whose serial number identifies the agent — closed by a photo of the token and the partner's count; INR in Delhi and Mumbai, or USD cash in Bangkok and

Singapore, are different agents at different rates.

D31 (L) — Wallet screening

Decision. `WALLET_SCREENING` per address and day for sending and receiving wallets (result, decision, who, when, evidence); `COLLECTION` gains `sending_address`, `wallet_screening_id` and `screening_skipped`; `COLLECTION_RECEIVING_ENDPOINT.risk_band`; `DESTINATION_APPROVAL.wallet_screening_id`. A screening decided accept_alternate lands only on an elevated-band endpoint; a reject is overridden only by Finance.

Rejected alternative. A static risk tier on the sender.

Why. NPL screens the sender's declared wallet before giving out the partner's receiving wallet, routes elevated-risk wallets to alternate wallets, verifies the hash on chain, and screens receivers' wallets before settlement. NPL: screening is a transaction-coordination activity, so it is an Operations action and Operations sees the result. Not every Evo sender accepts up-front screening.

D32 (M) — Single-use accounts and loss events

Decision. `COLLECTION_RECEIVING_ENDPOINT` and `RECEIVING_ENTITY_ACCOUNT` gain `usage` single_use with `reserved_for_deal_id` (and an expected amount on the endpoint); `LOSS_EVENT` on either side, by any method, for a partial or full amount, with the loss split across sender, NPL, partner and client (shares sum to the amount, Management approves); one reversing transaction (source LossEvent); `EXP_LOSS`; `COLLECTION.state` reversed. Cash found short at hand-over is a short confirmation topped up at the next settlement or by a separate payout, and a loss event only when unrecoverable.

Rejected alternative. A collection-only reversal borne by the sender in full.

Why. Local deposits use accounts the partner lends for one transaction; a frozen account can lose the funds. NPL: the loss can be partial, can be shared between NPL and the partner as well as the sender, can happen on the settlement side and by other methods, and single-use local accounts are used for settlement too.

D34 — Holiday calendar as dates

Decision. `HOLIDAY` under `PARTNER`: one row per non-business day of the partner's banking channel, with a scope (bank channel only, or the partner altogether) and a source. `PARTNER.holiday_calendar` becomes the name of the source calendar. Every "next business day" in the model — a leg's `conversion_due_date`, a settlement's `execution_due_date` — skips weekends in the partner's time zone and these rows; a reroute triggered by a holiday names one.

Rejected alternative. Keeping the calendar as a text label on PARTNER (the v4.0 shape).

Why. Since D29 the model computes due dates across the cutoff, and a label cannot be counted from. The FRS names the holiday calendar as a thing the system holds.

D35 — The quote package as a record

Decision. `QUOTE_PACKAGE` under `DEAL_GROUP`: one row per issue — the template version that rendered it, the channel, the rendered document reference, the printed sender-facing figures, `sent_at`, `sent_by` and a `content_hash`. The leg's `quote_hash` and `sent_at` are the package's. A re-issue (wording, another channel) is a new row that supersedes the earlier one; a new price is a new deal group. A quote-first leg is Sent only when a package has sent_at; freshness runs from the leg's market snapshot to sent_at, validity from sent_at.

Rejected alternative. Keeping only the facts behind the quote (rates, fees, timestamps, a hash) and not the artefact sent.

Why. Priced-once and the quote clocks rest on what the sender was actually shown; in a dispute the stored facts are not the document. The FRS's quote package had no home.

D36 — Message templates

Decision. `MESSAGE_TEMPLATE`: versioned wording of outgoing messages (quote, payment instructions, confirmation request, settlement advice, cash token), global or owned by a project, on a channel and in a language, with `printed_fields` — the model fields it may print. A quote template prints sender-facing figures only (sender rate, customer-facing fee and rate, amount out, validity), never partner rates, margins or costs. Management approves template changes as configuration.

Rejected alternative. Templates kept in the application layer with no record in the model.

Why. A quote package has to name what rendered it, and the rule that Operations never passes derived economics to a sender (invariant 9) is enforceable only if the template declares what it prints.

D38 — A CLIENT above PROJECT

Decision. `CLIENT`: the commercial relationship NPL contracts with and reports on — name, category (NPL's own grouping from the Project Context tab), legal entity and KYC references, relationship owner, status, and an optional `exposure_ceiling` on the sum resting at partners across its projects. `PROJECT` gains `client_id`: a project is one arrangement with a client. `CLIENT_POSITION` rolls up balances, entitlement, custody, earnings and rebates over the client's projects — a view, never stored (invariant 35). The per-project view CLIENT_BALANCE is renamed `PROJECT_BALANCE` so the two cannot be confused. Parties stay per project.

Rejected alternative. Keeping project = client and reporting across projects by hand; or moving senders, receivers and accounts up to the client now.

Why. NPL: one client runs several complex arrangements (BF, BF Sub and BF Withdrawal are one client; 66 Group appears in remittance and in the game reseller), so with project = client there was no way to report per client. Moving the party tables up would change attribution, the allow list and registrations, which is the kind of change that is expensive after launch; a client-wide sender registry can be added later as a reference.

D39 — A partner's methods are configuration

Decision. `PARTNER` gains `supported_methods` (crypto / bank / cash / local_deposit) and its `bank_cutoff_time` becomes nullable — empty for Jeton and Aquanow, which bind nothing. Invariant 36: a leg's collection method and a settlement's method must be among the partner's methods, and a wallet endpoint's network must already exist under the partner's vehicle before a collection may name it; adding either is a Management-approved configuration change. Networks are not declared: the vehicle's wallet endpoints already say them.

Rejected alternative. Deriving everything from the configured endpoints and rails.

Why. Cash and local deposits have no standing endpoint (D19, D32), so whether a partner offers them cannot be read from the configuration until a deal is attempted. NPL: Jeton has no cash or local methods and no cutoff; a partner record must be customizable, not hard-coded.

D40 — Bridge wallets and the wallet registry

Decision. `OWN_WALLET` gains `role` (reroute_contingency / own_desk / collection_bridge) and `networks`. Invariant 3 is restated with a third allowed case: a collection on a bridge wallet, forwarded to the leg's partner as a hop (a settlement line to the partner_transit receiver) within `custody_max_hours`; the custody view reports it with age. `COLLECTION_RECEIVING_ENDPOINT.own_wallet_id` may name a bridge as well as an own desk. New view `WALLET_REGISTRY`(network, address): every known address with its owner and role — partner wallets with risk band, NPL's wallets by role, receivers' wallets with their approvals, screened sender addresses with their decision — the FRS 'wallet registry', as a view over the four tables rather than a second copy.

Rejected alternative. Treating the bridge collection as a reroute each time; a stored registry table.

Why. NPL: partners have designated wallets for senders who cannot be screened up front, and where a partner lacks a network (Ali has no ERC-20) NPL collects on its own alternate wallet and forwards on a supported network, without the risk exposure. As written, invariant 3 forbade that routine flow. The FRS wallet registry had no single home.

D41 — Cash settlement evidence; recommend, then approve

Decision. `SETTLEMENT_LINE.token_ref` is the receiver's token, provided up front and relayed to the agent; the line gains `evidence_ref`, the agent's photo of the token after hand-over, relayed back to the receiver, whose confirmation closes the loop. `COLLECTION.token_ref` may be replaced by the partner before hand-over (a new agent), with the rate locked at quote unchanged. `FEE_DECISION` gains `recommended_by` (Finance) beside `approved_by` (Management) and `APPROVAL` gains `cover_bank_fee` and `raise_fee_cap`: a covered bank fee, or a markup above the cap after discussing with the customer, is reviewed by Finance and approved by Management. A partner rate below the limit warns and never refuses a quote. AED cash rounds to 100, USD cash in Dubai to 10.

Rejected alternative. Finance approving fee concessions on its own, as the generic one-off fee decision allowed.

Why. NPL: on settlement the receiver provides the token and the agent photographs it; covering a fee needs Finance's review and Management's approval; NPL may go above 2 % on THB after talking to the customer, so the system warns and Management decides on the cap.

D45 — Extending a quote's validity, as often as needed; honouring an expired rate

Decision. Replaces D42. `QUOTE_VALIDITY_EXTENSION`: one row per request — from and to validity, reason, approver, approval — any number per leg, each moving `DEAL.effective_valid_until` and re-issuing the `QUOTE_PACKAGE` at the same

price; Finance approves, Management is added when the new validity passes the partner's cutoff or the rate version's expiry. After expiry a new deal group is required; at the sender's request it may be priced on the earlier leg's market rate and partner rate version — `rate_reused_from_deal_id`, `rate_override_reason`, `rate_override_approved_by`, the approval `honour_expired_rate` — with the automated rate set aside and the grace drift limit not applied; the difference against the partner's actual rate posts to variance. Invariant 37 restated, invariant 39 new.

Rejected alternative. The once-only, within-grace extension of D42; refusing the original rate after expiry.

Why. NPL: an Evo sender may ask for a 1-hour quote to be held for 6 hours, in more than one request, and a sender whose quote has expired may still want the original rate — the system must allow that rate to be entered by hand on the new deal rather than refuse. The price of a leg stays write-once: the clock moves by approved rows, and the old rate is copied into a new leg on record, with its cost visible.

D37 — Names aligned with the FRS and NPL's words

Decision. Tables: `DISBURSEMENT` → `SETTLEMENT` (and _LINE, _RETURN; `disburse_policy` → `settlement_policy`); `INTRODUCER` → `MARKUP_SHARE_PARTY`, `REFERRAL_RULE` → `MARKUP_SHARE_RULE`, `REFERRAL_ACCRUAL` → `MARKUP_SHARE_ACCRUAL`, purpose `REFERRAL_PAYABLE` → `SHARE_PAYABLE`, ledger source Referral → MarkupShare, cost component referral merged into share; `SETTLEMENT_REGISTRATION` → `DESTINATION_APPROVAL`. Columns: `sender_share_pct` / `receiver_share_pct` → `sender_pays_pct` / `receiver_pays_pct`; `floor_pct` / `cap_pct` → `min_markup_pct` / `max_markup_pct`; `displayed_fee_basis` / `displayed_fee_pct` / `displayed_source_rate` → `customer_facing_fee_basis` / `customer_facing_fee_pct` / `customer_facing_rate`; `partner_markup_pct` → `partner_fee_pct` with `partner_pricing` values fee_on_market / fee_in_rate; `rate_basis` values market / agent_rate. Kept at NPL's request: `COLLECTION_SENDING_ENTITY`, `COLLECTION_RECEIVING_ENDPOINT`, `CONVERSION.residual_in`. Appendix A.1 carries the old names as aliases.

Rejected alternative. Keeping the technical names and showing NPL's words only as labels in the app and the documents.

Why. The FRS and NPL say settlement, not disbursement; NPL's fee table calls Jeton's rebate a 'referral commission', so 'referral' in the model pointed the wrong way and since D26 the party is usually a customer such as Raeen; NPL's word for a registration is 'approved'; the column names are the phrases NPL used when it answered the fee outlines ('sender pays', 'max markup', 'customer-facing calculation', 'fee on top of market', 'agent rate'). The earlier editions of the dependent documents still use the old names until they are regenerated.

D33 — Same-currency pass-through stays in scope

Decision. A `CURRENCY_PAIR` may have the same currency on both sides; the leg converts at rate 1 so the sender and receiver fee parts are captured at conversion like on any pair.

Rejected alternative. Dropping the dormant Evolution bank-transfer products.

Why. NPL: the fee is still applied because the currency is collected and then settled separately.

Amendment of 11 October, evening (same draft v5.7) — D46

An independent re-derivation of every calculation from the ERD and the Ledger Posting Design found five gaps between the model's definitions and the postings, all applied the same evening. **(1) Views over postings.** `PROJECT_BALANCE` had summed conversions' actual_out, which includes NPL's margin, and had no term for top-ups, make-goods, offsets, hops or re-raised promises; `GROUP_ENTITLEMENT` summed legs and lines. Both are now sums over postings: `LEDGER_POSTING.receiver_group_id` on every PAYABLE line, PROJECT_BALANCE = the DUE account balance per partner; invariant 7 restated. **(2) The two-invoice model.** `INVOICE.vendor_invoice_id` pairs a resale invoice with the vendor invoice it marks up; `DEAL_GROUP.invoice_amount` and `invoice_rate` (payout currency per invoice currency at pricing) carry the part collected and the stored base of every reserved amount; `INVOICE_BALANCE` is a receivable balance for a resale invoice and a payable balance over released lines for a vendor invoice; invariants 23 and 44 restated. **(3) FX_RESULT replaces FX_CLEARING**: an asset clearing account can never be closed under per-currency balancing, so NPL's own currency exchanges post to a profit-and-loss account per currency (currency sold a debit, currency bought a credit), valued through REPORTING_VALUE; a cash-rounding surplus is revenue, never a forex result. **(4) Invariant 16** restated as two lists, client → company and company → client. **(5) Invariant 42** gains the target-out rule: the fee is a markup on the target, amount_in = target × (1 + sender_pays_pct) ÷ rate, NPL's own convention. **(6)** `PARTNER_REBATE_RULE.basis` keeps pct_of_amount only.

Amendments of 9 October (same draft)

D15. NPL confirmed the PARTNER_ENTITY rename. **D27.** `pct_of_partner_fee` is retired; `basis` + `pct` carry the rebate rule. **D29.** The bank cutoff moves to `PARTNER`: it is the partner's clock, like its rate expiry and holiday calendar, whichever

project uses it. Later the same day NPL added that the cutoff governs two events — locking the rate and initiating the outgoing bank settlement — on the pairs that rely on the banking channel, and that it can be extended (30–60 minutes, partner agrees) or closed early (Fridays, pre-holiday): `PARTNER_CUTOFF_OVERRIDE` holds those per-day or weekday changes, the effective cutoff is stamped as `cutoff_at` on the leg and on the settlement, and `SETTLEMENT.execution_due_date` mirrors the leg's conversion due date for bank rails. **D30** stands as first drafted: the cash city is `DEAL.cash_location` and `PARTNER_RATE_VERSION.location`. An amendment making the city a side of the currency pair (a CASH_LOCATION table) was applied and then withdrawn the same day at NPL's request. Stale wording from before D23, D25 and D26 was corrected in the notes, walkthroughs, invariant 4 and the FRS reconciliation.

New tables (v5.3)

Table	Holds
EARNINGS_OFFSET	NPL's converted margin at one partner swapped for the same value of the client's unconverted money at another, at the partner's rate of the day; recovers earnings receivables; Finance requests, Management approves (D46).
LOSS_RECOVERY	One recovered share of a loss event after the whole loss was booked to NPL: partner, sender or client, amount, state, reference (D46).
CLIENT	The commercial relationship above PROJECT; several projects per client; optional exposure ceiling across them (D38).
FEE_TIER	A volume tier of a fee structure: minimum calendar-month volume, currency, pct, optional split (D21).
DEAL_FEE_TIER	The tier portions of one leg — two when the leg crosses a threshold; amounts sum to amount_in (D21).
INVOICE	A vendor obligation in its own currency, owned by the project, paid by one or more deal groups and discharged line by line (D24).
WALLET_SCREENING	A screening of one wallet address on one day — sender or receiver side — with result, decision, who and evidence (D31).
LOSS_EVENT	Money lost after verification or release, on either side, partial or full, with the loss split across sender, NPL, partner and client (D32).
PARTNER_CUTOFF_OVERRIDE	A one-day or weekday change to a partner's bank cutoff: an agreed extension or an early close (D29, amended 9 October).
HOLIDAY	A non-business day of a partner's banking channel, with scope and source; the dates behind every next business day (D34).
QUOTE_PACKAGE	What was sent to the sender for a deal group, per issue: template version, channel, printed figures, sent_at, hash (D35).
MESSAGE_TEMPLATE	Versioned wording of outgoing messages with the fields it may print; quote templates are sender-facing only (D36).
SYSTEM_SETTING	NPL-wide Management-approved settings: reporting currency, staleness limit, operating time zone (D43, D44).
QUOTE_VALIDITY_EXTENSION	One approved extension of a quote's validity; any number per leg (D45).
ROLE	User roles as configuration: code, name, whether it sees economics (D44).
ROLE_RIGHT	Per role and controlled action: may initiate, may approve, role added on escalation (D44).

Changed tables (v5.3)

Table	Change
INVOICE	+ kind vendor / resale, + sender_id — NPL-GR's two invoices per deal, posted once as cost and revenue (D46); + vendor_invoice_id (amendment).
DEAL_GROUP	+ invoice_amount, invoice_rate — the part of the resale invoice collected and the pricing rate (D46 amendment).
LEDGER_POSTING	+ receiver_group_id on promise lines (D46 amendment).
PARTNER_REBATE_RULE	basis pct_of_amount only; pct_of_partner_fee dropped (D46 amendment).
EARNINGS_RECEIVABLE	+ recovered_by_offset_id (D46).

Table	Change
PARTNER_REBATE_RULE	+ payout_currency (D46).
CURRENCY_PAIR	amount_rounding + exact — bank transfers carry cents; unit rounding is for cash (D46).
CONVERSION	residual_in is the margin kept in the collection currency, EARN_GROSS recognised in that currency (D46).
SETTLEMENT	release discharges the promise; a return or a short confirmation re-raises it; INTRANSIT retired (D46).
CONFIRMATION	posts only when short (D46).
FEE_STRUCTURE	fee parts on the principal at the basis rate, never compounded (D46).
LEDGER_ACCOUNT	purposes VAR_CONVERSION / VAR_CUTOFF / VAR_RATE_HONOUR / VAR_FX_TIMING (was VARIANCE), LOSS_RECEIVABLE, MAKEGOOD, LOSS_PAYABLE, RESELL_REVENUE, RESELL_COST, VENDOR_PAYABLE, RESALE_RECEIVABLE, REMIT_CLAIM, FX_RESULT (amendment: replaces FX_CLEARING); INTRANSIT retired (D46).
LEDGER_TRANSACTION	sources + LossRecovery, Invoice, EarningsOffset (D46).
LEDGER_POSTING	cost components + resale; balanced per owner as well as per currency (D46).
APPROVAL	actions + earnings_offset; ROLE_RIGHT row to match (D46).
PROJECT	+ client_id — one arrangement with a CLIENT (D38).
PARTNER	+ supported_methods; bank_cutoff_time nullable (D39).
OWN_WALLET	+ role, networks — a collection bridge is a third custody case (D40).
SETTLEMENT_LINE	+ evidence_ref — the agent's token photo on a cash payout; token_ref is the receiver's token (D41).
FEE_DECISION	+ recommended_by — Finance recommends, Management approves (D41).
DEAL	+ effective_valid_until; + rate_reused_from_deal_id, rate_override_reason, rate_override_approved_by (D45).
APPROVAL	actions + extend_quote_validity (repeatable), honour_expired_rate (D45).
CURRENCY_PAIR	project_id nullable — system reporting pairs (D43).
CLIENT	exposure_ceiling stated in the reporting currency (D43).
USER	role → role_id, a ROLE row (D44).
CURRENCY_PAIR	+ scope project / platform (D44).
FEE_OVERRIDE	+ sender_pays_pct, receiver_pays_pct (D20).
FEE_STRUCTURE	+ customer_facing_fee_basis, customer_facing_fee_pct (D22); owns FEE_TIER (D21); may scope a markup-share or rebate rule (D26, D27).
DEAL	+ fee_override_id, fee_tier_id, tier_volume_basis (D20, D21); + rate_locked_at, cutoff_at, conversion_due_date, conversion_deferred_reason (D29); + cash_location (D30); quote_hash and sent_at are the quote package's (D35).
DEAL_GROUP	owns QUOTE_PACKAGE (D35).
SETTLEMENT	+ cutoff_at, execution_due_date — a bank-rail settlement approved after the cutoff goes out the next business day (D29, amended 9 October).
DEAL_GROUP	+ customer_facing_fee_pct, customer_facing_rate (D22); + invoice_id (D24).
PARTNER	+ bank_cutoff_time, cutoff_timezone (D29, amended 9 October: moved here from PARTNER_CONFIG); owns PARTNER_CUTOFF_OVERRIDE; holiday_calendar is now the calendar's name and the dates are HOLIDAY rows (D34).
PARTNER_CONFIG	+ is_own_desk (D23).
PARTNER_PAIR	+ needs_bank_lock (D29).
COLLECTION_RECEIVING_ENDPOINT	+ own_wallet_id (D23); + risk_band (D31); + usage, reserved_for_deal_id, expected_amount (D32).

Table	Change
RECEIVING_ENTITY_ACCOUNT	+ usage, reserved_for_deal_id (D32).
CURRENCY_PAIR	amount_rounding becomes a menu; + rounding_unit; + cash_rounding, the same menu for the collected cash amount (D25); from_currency may equal to_currency (D33).
CONVERSION	rounding_residual signed (D25); rate 1 on a same-currency pair (D33).
MARKUP_SHARE_RULE	basis + share_of_net_markup; + fee_structure_id, party_kind, settlement_mode (D26).
MARKUP_SHARE_ACCRUAL	+ direction, period, received_amount; states accrued / reconciled / settled (D26).
PARTNER_REBATE_RULE	+ basis, pct, fee_structure_id, sender_id; – pct_of_partner_fee (D27).
PARTNER_REBATE_ACCRUAL	+ statement_ref; deal_id stays required (D27).
PARTNER_RATE_VERSION	+ quoted_rate, quoted_direction (D28); + location (D30); rate derived.
BALANCE_CONVERSION	+ rate_locked_at, conversion_due_date (D29).
COLLECTION	+ sending_address, wallet_screening_id, screening_skipped (D31); + token_ref, agent_contact_ref, counted_at (D30); state + reversed, cash sub-states (D30, D32).
SETTLEMENT_LINE	+ invoice_id, obligation_discharged, discharge_rate, discharge_market_rate_id (D24); + token_ref (D30).
DESTINATION_APPROVAL	+ wallet_screening_id (D31).
LEDGER_ACCOUNT	purposes + EARN_REBATE (D27), SHARE_RECEIVABLE (D26), EXP_ROUNDING (D25), EXP_LOSS (D32).
LEDGER_TRANSACTION	source types + LossEvent (D32).
LEDGER_POSTING	cost components + rebate, share, fx_timing, cutoff_timing, loss.
APPROVAL	actions + enable_own_desk (D23), approve_loss_split (D32), override_screening_reject (D31).

Views and invariants (v5.3)

Item	Change
PROJECT_BALANCE	Restated: the balance of the DUE account per partner, over postings (D46 amendment).
GROUP_ENTITLEMENT	Restated: Σ PAYABLE postings by receiver_group_id (D46 amendment).
INVOICE_BALANCE	Restated by kind: a resale invoice's receivable balance over collected groups; a vendor invoice's payable balance over released lines (D46 amendment).
Invariants 7, 16, 23, 42, 44	Restated by the D46 amendment: entitlement conservation over postings; ownership movements in both directions; the two-invoice case; the target-out rule; the cash-rounding surplus as revenue.
RESELLER_MARGIN	New: resale invoice – the vendor invoice it marks up (vendor_invoice_id), in the invoice currency; equal to the two postings and untouched afterwards (D46).
REMITTANCE_MARGIN	New: earnings – partner cost ± forex – fees on one deal's postings, in the converted currency (D46).
NPL_PROFIT	New: remittance earnings + reseller earnings, each callable alone and together, per currency; one figure across currencies through REPORTING_VALUE (D46).
Invariants 16, 25, 26, 28, 39	Restated: the four ownership events and per-owner balancing; accruals posted by the conversion; the variance accounts named (D46).
Invariants 40–44	New: the promise discharged on release and every released line accounted for; a pool negative only until the next conversion squares it; fee parts on the principal; an earnings offset neutral for the client at the partner's rate; the reseller revenue intact by construction (D46).
PROJECT_BALANCE	+ pending: the expected_out of rate-locked legs not yet collected or converted; removed on void; never a posting (D29).
CUSTODY	Includes own-desk client balances with their age (D23).
INVOICE_BALANCE	New: invoice amount – Σ obligation_discharged over confirmed lines (D24).
PROJECT_MONTH_VOLUME	New: Σ amount_in of the project's forward legs in the calendar month, in the tier currency; read at quote (D21).

Item	Change
CLIENT_POSITION	New: balances, entitlement, custody, earnings and rebates summed over a client's projects, compared with the client's exposure ceiling (D38).
WALLET_REGISTRY	New: every known wallet address with its owner and role, over the partner, own, receiver and screened-sender tables (D40).
REPORTING_VALUE	New: any per-currency figure converted into the reporting currency at the latest market rate, with the rate, its time and a stale flag; display only (D43).
CUSTODY	Includes bridge collections awaiting their hop (D40).
Invariant 3	Restated again: three custody cases — approved reroute, own-desk project, collection bridge forwarded within custody_max_hours (D40).
Invariant 9	Restated: approver ≠ initiator, and the approver's role has may_approve for the action in ROLE_RIGHT (D44).
PROJECT_BALANCE	Renamed from CLIENT_BALANCE: the client's money resting at one partner for one project; the client-level figure is CLIENT_POSITION (D38).
Invariant 3	Restated: custody is allowed under an approved reroute or in an own-desk project; the custody view reports both (D23).
Invariants 20–39	New (invariant 32 covers D33; 33 business days from HOLIDAY; 34 the quote package and its template; 35 client-level sums and the client exposure ceiling; 36 a partner offers only what its record says; 37 validity extensions by approved rows, price unchanged; 38 converted figures are display-only estimates; 39 an expired rate honoured only on a new deal by an approved override): override split, tier choice and split, customer-facing figures derived, invoice discharge bounded, signed rounding residual with an owner, share base and Σ shares, rebate stream and deal-bound accruals, rate derived from the quoted rate, conversion and settlement due dates from the effective cutoff with its overrides, rate uniqueness per location and cash count, screening ↔ endpoint band, single-use references and loss shares, same-currency conversion at rate 1.

Dependent documents

Document	State against this draft
Ledger Posting Design v5.7 and Ledger Patterns Reading Companion v5.7	Regenerated with this draft: the chart of accounts and 59 worked transactions covering every pattern of D46, each with a plain-language explanation and its arithmetic in the companion. Numbered with the ERD edition.
Lifecycle & Approvals v1.1 → v1.2	Rate-locked and awaiting-cutoff sub-states with the deferral; loss event path with Management approval; cash collection steps (token → hand-over → count); screening before Collecting as an Operations action; enable_own_desk.
Roles & Visibility Matrix v1.0 → v1.1	Screening fields in the Operations group; rebate, party share and loss split in the economics group; a party-facing statement shows the share only, never the rebate.
Calculation Specification v1.0 → v1.1	Tier split and blended fee; customer-facing rate; rate-direction normalisation; rounding menu; same-currency conversion at rate 1; acceptance vectors from NPL's real figures (market 0.86050 / Ali 1 ÷ 1.1738; GDC 86,594; Raeen 41,520.17; THB 17,000,000; USDT 2,598 → 2,600).
Configuration Schema v1.0 → v1.1	Every new column and value set; the NPL-GR own-desk project; rebate rule basis and scope; cutoff times; cash locations; single-use endpoints.
ERD Reading Guide v1.0 → v1.1	The new boxes on 1a, 1b, 3a and 3c, the split of the deal diagram into 3a (deal, collection) and 3b (conversion, reroute) with settlement now 3c, and the decisions behind them.
Fee Outlines v0.2 → v0.3	Pink cells resolved (Ad Hoc INR 7 % total; Raeen 3.2 %; INR rounding); 'needs a system change' notes become 'configured in v5.3'; the tier split shown on pattern 5.
Model Gaps & Proposals v1.0 → v1.1	The corrections of 9 October (H simplified, J pending balance, L Operations, M generalised, B split) and the status 'applied in v5.3' on all thirteen.
ERD Review — Decision Record	D20–D33 recorded (9 October 2026).
data/erd.v5.json, interactive map, knowledge base	The data file, the map and the knowledge base are at 5.7 with this draft.

Earlier revisions — v5.0 to v5.2

D13 — No return group

Decision. `sender_return` and `partner_transit` receivers belong to no group. Attribution of a return or hop leg is inherited from its parent leg. Offsets are defined between a leg's attributed group and the group of a counterparty receiver.

Rejected alternative. A dedicated group of kind = return per project holding the sender_return receivers (ERD v4.0 §6; config schema rule "one default and one return group").

Why. Groups exist for entitlement and offset accounting; refund exclusion is a receiver-kind rule, not an entitlement rule, and `RECEIVER.kind` already enforces it. The return group (a) made every project have at least two groups, so the Understanding document's "a project with a single group never has an offset" was false by construction; (b) classified every refund as a cross-group payout under the v4.0 offset definition, triggering Management approval and offset aging; (c) left the return group's entitlement negative after each refund, breaking invariant 7, unless a reattribution path was added. Removing it restores the Understanding document's §3.3 and §6.3 wording without change. The return group entered during the v4.0 write-up; decision D3 of the review record relied on it and is amended there.

Changed tables

Table	Change
`RECEIVER_GROUP`	− kind. A group is a group; exactly one is_default per project (D13).
`RECEIVER`	group_id required when kind = counterparty, null otherwise. sender_return and partner_transit receivers never appear on forward deals or allow lists (validation on kind) (D13).
`DEAL`	receiver_group_id: set at quote on the first forward leg; a return leg inherits it from its parent leg, leg n of a route from leg n−1 (D13).
`SETTLEMENT_LINE`	A line paying a group-less receiver debits the paying leg's group and is never an offset (D13).
`RECEIVING_ENTITY_ACCOUNT`	+ account_kind (bank / wallet); details_ref → bank_details_ref (bank accounts only); + network, + address (wallets only). Write-time rule: a bank account has bank_details_ref and a bank rail; a wallet has network and address and a crypto rail matching the network (D14).

D14 — Explicit wallet fields on receiving-entity accounts. Rejected: keeping wallet details inside the opaque details_ref. Why: the collection side already stored network and address explicitly; the payout side did not, so addresses could not be validated, checked against the rail, or queried across partners. Bank details stay external by reference, as before.

D15 — Partner entity and collection endpoint

Decision. `FACILITATING_ENTITY` is renamed `PARTNER_ENTITY`: the partner's legal vehicle whose name appears on paperwork, the mirror of `RECEIVING_ENTITY` on the client side. `COLLECTION_RECEIVING_ENTITY` (bank-in) and `WALLET` (crypto-in) merge into one `COLLECTION_RECEIVING_ENDPOINT` table owned by a partner entity, with kind = bank / wallet and the field shape of `RECEIVING_ENTITY_ACCOUNT` (rail required only on active bank endpoints, see D18). `COLLECTION` points at one endpoint (`collection_receiving_endpoint_id`, null for cash) instead of the hand-rolled endpoint_type + endpoint_id pair. NPL confirmed the rename on 9 October 2026; the FRS term "Facilitating Entity" stays in Appendix A.1.

Rejected alternative. Keeping two endpoint tables, one with a legal owner (wallets under a vehicle) and one without (bank accounts directly under the partner config).

Why. A bank account is always in some legal entity's name, so the v4.0 split left the schema unable to say which vehicle owns a bank-in account, while every wallet knew its owner. One endpoint table under the vehicle fixes the asymmetry, lets the system check that the account a sender paid into belongs to the vehicle stamped on the deal, removes a polymorphic pair from `COLLECTION`, and makes the partner side read exactly like the client side: config → entity → endpoint.

Table	Change
`PARTNER_ENTITY`	renamed from FACILITATING_ENTITY; − bank_details_ref (moved to its endpoints).
`COLLECTION_RECEIVING_ENDPOINT`	new; replaces COLLECTION_RECEIVING_ENTITY and WALLET. partner_entity_id, kind (bank / wallet), currency, bank_details_ref (bank), network + address (wallet), status.

Table	Change
COLLECTION	endpoint_type + endpoint_id → collection_receiving_endpoint_id (null for cash).
DEAL	facilitating_entity_id → partner_entity_id.
Invariant 6	adds: a collection's endpoint must belong to the leg's partner entity.

v5.2 — decisions of the review of the v5.1 draft (8 October 2026)

D16 — Registration payout details follow the account's kind

Decision. DESTINATION_APPROVAL gains network and address (nullable). bank_details_ref ("as paid by this partner") is set only when the registered account's account_kind is bank; for a wallet account the registration carries the network and address as the partner registered them, and the bank field stays empty. The registration remains the partner's view of the destination; RECEIVING_ENTITY_ACCOUNT remains NPL's.

Rejected alternative. Keeping a bank-only field on wallet registrations (the v5.1 shape).

Why. After D14 and D15 every other destination record is kind-aware; a wallet registration had no field that fitted, so an approved wallet destination either carried no details or carried an address in a field named for bank details. Storing the partner's copy also lets the system compare NPL's record with the partner's before paying to a wallet, where a wrong address is unrecoverable.

D17 — SETTLEMENT_LINE.deal_id

Decision. SETTLEMENT_LINE gains deal_id, a nullable foreign key to DEAL: required when the paid receiver is group-less (sender_return, partner_transit), optional on counterparty lines. New invariant 19. GROUP_ENTITLEMENT reads the paying leg's group from it.

Rejected alternative. Leaving the rule in prose ("a payout to a group-less receiver debits the paying leg's group").

Why. A SETTLEMENT pools many deals, and neither it nor its lines named one; the D13 rule and the view that depends on it could not be computed from stored columns, and invariant 7 could not be checked mechanically. The same column will carry the per-line discharge facts of the scenario in which a EUR invoice is paid in USDT in parts.

D18 — Nullable rail on COLLECTION_RECEIVING_ENDPOINT

Decision. The endpoint gains rail, nullable. Null means "not yet known", never "does not matter": a bank endpoint cannot become active — and no COLLECTION or payment instruction may reference it — until rail is set; a wallet endpoint may leave rail null (its network already names the road), and when rail is set on a wallet it must match network (mirrors D14).

Rejected alternative. A NOT NULL rail, which forces a guessed value when a partner sends incomplete details; or no rail at all (the v5.1 shape), which made the "same field shape as RECEIVING_ENTITY_ACCOUNT" claim inexact.

Why. The road matters on the collection side as much as on the payout side: the same EUR account reachable by SEPA or SWIFT needs different instructions and arrives with different fees; local rails carry transaction limits (UPI caps, RTGS minimums); and a wrong rail recorded with confidence causes the failures a blank cannot.

D19 — Endpoint kind must match the leg's collection method

Decision. A write-time rule, added to invariant 6: a collection endpoint's kind must match the leg's collection_method — wallet for crypto-in, bank for bank-in, and cash-in has no endpoint.

Rejected alternative. None; the rule was missing.

Why. After D15 the fact "how does the money arrive" lives in two places — DEAL.collection_method and the endpoint's kind — and nothing stopped them disagreeing silently.

Changed tables (v5.2)

Table	Change
DESTINATION_APPROVAL	+ network, + address (wallet accounts only); bank_details_ref only when the account's kind is bank (D16).
SETTLEMENT_LINE	+ deal_id — nullable FK to DEAL; required when the paid receiver is group-less, optional on counterparty lines (D17).

Table	Change
COLLECTION_RECEIVING_ENDPOINT	+ rail — nullable; required before a bank endpoint becomes active; on a wallet, must match network when set (D18).
COLLECTION	No column change: may reference only an active endpoint whose kind matches the leg's collection_method (D18, D19).

Views and invariants

Item	Change
GROUP_ENTITLEMENT	Sums client_net over converted legs of both types; a payout line debits the paid receiver's group (counterparty) or the group of the leg named by its deal_id (group-less receivers) — computable from stored columns (D13, D17).
OFFSET	Entitlement − paid per group, arising only when a counterparty receiver is paid from a leg attributed to another group.
Invariant 6	Adds: a collection's endpoint must belong to the leg's partner entity (D15); its kind must match the leg's collection_method (D19); a COLLECTION may reference only an active endpoint, and a bank endpoint is active only with rail set (D18).
Invariant 7	Restated: return and hop legs preserve conservation by inheriting the parent leg's group; since v5.2 every payout line resolves to a group, so the invariant is checkable mechanically (D17).
Invariant 17	Restated: offsets only between a leg's attributed group and a counterparty receiver's group; a single-group project never has an offset.
Invariant 18	group_id iff kind = counterparty; forward legs and the allow list target counterparty groups and receivers only; return and hop legs inherit attribution.
Invariant 19	New: a line paying a group-less receiver carries deal_id; the paying leg's group is a stored fact, never inferred (D17).

Editorial corrections in v5.2

- Invariant 18's note now says the allow list targets counterparty *groups and receivers*, not receivers alone.
- The D13 write-up no longer claims the return group appeared in no decision-record entry: decision D3 of the review record did place sender_return receivers in a return group; the record amends D3 and carries D13–D19.
- The dependent-documents table is refreshed: the data file is already `erd.v5.json` and the decision record carries D13–D19.
- The third v5.1 change (D15) is dated 8 October 2026; this baseline is an internal New XP document from which the client deliverables are derived.

Dependent documents as updated for v5.2

Document	Change
Configuration Schema Specification v1.0	Replace "one default and one return group" with "exactly one default group"; add "every counterparty receiver names a group; no other kind does". Partner section: entities own endpoints; endpoint kind decides which fields are required; rail nullable with the activation rule (D18); registration fields follow the account's kind (D16). Re-validate the FRS Client 1 example — it should pass with one group.
Lifecycle & Approvals v1.0	cross_group_payout must not fire on return legs or hop legs; endpoint activation gated on rail (D18); collection verification checks the endpoint's kind against the leg's collection_method (D19).
Ledger Posting Design v1.0	No account changes. Confirm the return-leg example posts the payout against the parent leg's group entitlement; payout postings may cite the line's deal_id (D17).
Calculation Specification v1.0	No change.
ERD Review — Decision Record	D13–D19 recorded (8 October 2026).
Understanding document v1.1	Nothing to apply; it never had a return group.
`data/erd.v5.json`, interactive map, knowledge base	Updated to version 5.2 with this draft (fields, views, invariants, notes, vocabulary).

Carried over from v4.0 unchanged

The 12 new tables, 16 changed tables and 3 removals listed in v4.0 §6 (MARKUP_SHARE_PARTY, RECEIVER_GROUP, RECEIVING_ENTITY_ACCOUNT, MARKUP_SHARE_RULE, PARTNER_REBATE_RULE, DEAL_GROUP, BALANCE_CONVERSION, ENTITLEMENT_REATTRIBUTION, SETTLEMENT_RETURN, FEE_DECISION, MARKUP_SHARE_ACCRUAL, PARTNER_REBATE_ACCRUAL; the removal of RECEIVER_ENTITLEMENT, withhold_earnings and the ADVANCE purpose) all stand. Of the small gaps noted in the engagement record, cash rounding per currency is closed by D25 and the partner's time zone by D29 (cutoff_timezone); EXCEPTION entity, PARTNER_STATEMENT (a header for the monthly rebate statement would sit above PARTNER_REBATE_ACCRUAL.statement_ref), configuration versioning, SENDER_CREDIT.applied_to_deal_id and DEAL_GROUP.estimate_out remain open.

7 · Open questions

1 of the questions carried from the Understanding document remain open; the rest were answered by NPL in the review of the Understanding document or settled in the ERD reviews of 6–9 October 2026 and are reflected in this draft.

Still open

#	Question	Blocks	Status
9	FRS figure reconciliation (1,208 vs 1,225; Mode 1 example)	Golden cases	NPL: Needs clarification. To be discussed.

Answered and reflected in v5.7

#	Question	Answer	In the model
1	Collect-first: must SENDER_RECEIVER_ALLOW pre-exist for attribution; handling of inbound matching no sender	NPL: Yes, sender-receiver mapping should be done in configuration. When the transaction is created, the sender is picked from a list.	v4.0: write-time rule that a collect-first deal needs an existing allow entry; the allow list now targets receiver groups.
2	Withholding-eligibility criteria on rerouted deals	Review (7 Oct): NPL's cut is always captured at conversion, so it is never at risk from a return or a forward. If forwarding happens before conversion, whichever partner converts captures it.	v4.0: the eligibility question dissolves. forwarding_policy is forward_full or forward_and_recover_dues; withhold_earnings is removed. Ownership flips at one event only.
3	Post-collection reroute (funds stuck at a partner): transfer, offset, or refund-and-requote	NPL: The partner can send the funds to NPL (and NPL decides where to send) or the partner can refund the sender.	v4.0: REROUTE.outcome via_own_wallet / refund_sender; a refund is a return leg paying the sender_return receiver.
4	Receiving-entity constraints (currency, rail) narrowing settlement choice	NPL: Receiving entities have specific bank accounts with currency and rail restrictions, for example EUR via SWIFT and EUR via SEPA.	v4.0: RECEIVING_ENTITY_ACCOUNT; registrations and payout lines point at the account.
5	Any deliberate partial-settlement / reserve policy	NPL: Yes, possible. In some projects the client's funds sit until they are ready to receive them, within an exposure threshold NPL sets by appetite.	v4.0: PROJECT.settlement_policy to_zero / hold_allowed with exposure_ceiling as the cap; invariant 4 restated.
6	Receiver-side fee timing per project	NPL: Receiver-side fee should be applied at the time of conversion, otherwise the balance will be incorrect.	v4.0: default at_conversion; CONVERSION.fee_receiver_part; SETTLEMENT_LINE.receiver_fee_applied becomes exceptional.
7	Network-fee policy on refunds	NPL: Set at project level with per-transaction override (one 100 USD fee was repaid in the next settlement).	v4.0: PROJECT.network_fee_policy plus FEE_DECISION for the override.
8	Whole-unit conversion rounding rule	Review (7 Oct): fractions are dropped to whole units at conversion, per transaction (30.152 → 30). The dropped fraction stays client money, carried in the balance and paid out when it reaches a whole unit. Cash collections round to the nearest 100 / 500 / 1000 by currency.	v4.0: CONVERSION.rounding_residual; CURRENCY_PAIR.amount_rounding and cash_rounding_unit; settle to less than one unit; SHORTFALL no longer takes rounding. Superseded in v5.3 by D25: a rounding menu per pair, signed residual, EXP_ROUNDING for round-ups.
10	AQN-LT-Sub "settled by offset against transactions outside the system"	NPL: This is NPL's wallet (pass-through at collection or on the settlement side). AQN is Aquanow, a crypto exchange; LT Sub is the account name.	v4.0: OWN_WALLET.provider and account_label; pass-through is a two-leg route or a reroute via the own wallet.

#	Question	Answer	In the model
11	Reference workbooks per project archetype	Fee Outlines v0.2 (8 October 2026): every line of NPL's fee table as a fill-in outline with a worked calculation per transaction pattern, answered by NPL with the Project Context and Payment methods tabs.	v5.3: decisions D20–D33 were built from it; the workbook is the acceptance reference for the fee engine.
12	FRS P1 / P3 vs rerouting and multi-partner use — amend FRS or restrict scope	NPL: Check the added explanations (partner types and channels; pass-through and double conversions in §7.1.1). Ask again if still unclear.	v4.0: two-leg routes are deal groups with two legs and a partner_transit receiver for the hop; FRS principles generalised, scope kept.
13	Advance funding source	Review (7 Oct): never. NPL does not pay a receiver before the client's money has arrived.	v4.0: LEDGER_ACCOUNT purpose ADVANCE removed; the advance approval and the "advance" ownership-flip event removed.
14	Review point: entitlement split on a balance conversion	Review (7 Oct): chosen group, not proportional. The conversion is triggered by a specific payout.	v4.0: BALANCE_CONVERSION.receiver_group_id; excess over the group's source-currency entitlement is booked as an approved offset.
15	Review point: refunds to a third party at the sender's request	Review (7 Oct): allowed by exception only. Default is the sender's own return receiver; any other destination needs Management approval with a reason and a pre-registered approved receiver.	v4.0: invariant on return legs; APPROVAL action refund_to_third_party.
16	Review point: recover dues on a reroute	Review (7 Oct): yes, NPL nets what a partner owes it against funds about to be forwarded to that partner.	v4.0: forwarding_policy forward_and_recover_dues, recovered_amount, partner_agreement_ref; postings own wallet cash ← earnings receivable, no income.
17	Review point: legs of a route share one client-facing quote	Review (7 Oct): yes, a deal group. The sender saw one price and one reference; margin across legs is the group quote minus the partner conversions.	v4.0: DEAL_GROUP; every deal belongs to a group, a single-partner deal being a group of one leg.
18	Review point (v5.0): does a refund need its own receiver group?	Review (7 Oct, New XP): No. Refund exclusion is a receiver-kind rule; a dedicated return group made every project multi-group (so "a single-group project never has an offset" was false by construction), classified every refund as a cross-group offset needing Management approval, and left its entitlement negative after each refund, breaking invariant 7.	v5.0 (D13): sender_return and partner_transit receivers are group-less; return and hop legs inherit the parent leg's attribution; offsets are defined only against counterparty groups. New invariant 18.
19	Review point (v5.1): restructure the partner collection side — rename FACILITATING_ENTITY, merge the endpoints?	Review (8 Oct, New XP): yes. A bank account is always in some legal entity's name, so the v4.0 split could not say which vehicle owned a bank-in account while every wallet knew its owner. The rename to PARTNER_ENTITY was confirmed by NPL on 9 October 2026; the FRS keeps "Facilitating Entity".	D15: FACILITATING_ENTITY → PARTNER_ENTITY (bank_details_ref moved to its endpoints); COLLECTION_RECEIVING_ENTITY and WALLET merge into COLLECTION_RECEIVING_ENDPOINT (kind bank / wallet, same shape as RECEIVING_ENTITY_ACCOUNT); COLLECTION points at one endpoint (null for cash); invariant 6 adds that a collection's endpoint must belong to the leg's partner entity.

#	Question	Answer	In the model
20	Review point (v5.3): can the model express NPL's fee practice as described in the fee table, NPL's workbook answers and the Project Context / Payment methods tabs (gaps A–M)?	NPL (8–9 Oct): all thirteen confirmed with parameters — receiver part fixed (A); calendar-month tiers, a crossing deal split across tiers (B); displayed 1 % is a BF-only rule (C); run NPL-GR inside NPLify as an own desk (D); invoice tracking wanted (E); rounding menu, round up to 1,000 for Asian currencies (F); markup shared = fee – partner fee, several parties, party holds the funds, rebate NPL's alone (G); rebates reported apart, statement covers NPLify deals only (H); Ali quotes inverted (I); NPL absorbs the next-day rate difference, pending converted value shown from the lock (J); cash by token, rates by city (K); screening is an Operations action (L); losses partial and shared, both sides, any method (M); same-currency pass-through stays in scope.	v5.3, D20–D33: FEE_TIER, DEAL_FEE_TIER, INVOICE, WALLET_SCREENING, LOSS_EVENT; views INVOICE_BALANCE, PROJECT_MONTH_VOLUME; invariants 20–32 and invariant 3 restated; purposes EARN_REBATE, SHARE_RECEIVABLE, EXP_ROUNDING, EXP_LOSS.
21	Review point (v5.3): four FRS terms with no home in the ERD — wallet registry, holiday calendar, message template, quote package.	Review (9 Oct, NPL + New XP): wallet registry = COLLECTION_RECEIVING_ENDPOINT of kind wallet (D15). The other three are added: the holiday calendar was a label and could not drive 'next business day'; the quote package is the evidence behind priced-once and the quote clocks; the template is where the sender-facing-only rule on messages is enforced.	v5.3 D34 HOLIDAY, D35 QUOTE_PACKAGE, D36 MESSAGE_TEMPLATE; invariants 33–34; quote-first walkthrough gains the package step.
22	Review point (v5.4): project = client made client-level reporting impossible when one client runs several arrangements (BF, BF Sub, BF Withdrawal; 66 Group in remittance and in the game reseller).	NPL (9 Oct): add a parent clients table. New XP: yes — CLIENT above PROJECT with a roll-up view; parties stay per project for now (a shared sender registry can be added later without breaking attribution).	v5.4 D38: CLIENT (name, category, legal entity, KYC, relationship owner, optional exposure ceiling); PROJECT.client_id; CLIENT_POSITION view; invariant 35.
23	Review point (v5.4): NPL's answers to the operational questions of the scenario catalogue — alternate wallets, partner methods, cutoff scope, cash settlement token, fee cover, local deposits.	NPL (9 Oct, evening): partners have designated wallets for unscreened senders and NPL bridges unsupported networks on its own wallet; a partner's methods are configuration; the cutoff exists only for Ali-type partners and only on bank-side pairs; on a cash payout the receiver issues the token and the agent photographs it; a cash deposit into a bank account is a local payment; Finance recommends and Management approves fee concessions.	v5.4 D39 PARTNER.supported_methods, cutoff nullable, invariant 36; D40 OWN_WALLET.role / networks, invariant 3 restated, WALLET_REGISTRY view; D41 SETTLEMENT_LINE.evidence_ref, FEE_DECISION.recommended_by, approval actions cover_bank_fee and raise_fee_cap.
24	Review point (v5.4): a system currency for reporting — aggregated collection volumes, earnings and balances across currencies, per project or overall.	NPL (9 Oct): a display layer that converts each currency figure at the current market rate into one reporting currency, for project-specific or overall volumes, earnings and current balances; estimated figures.	v5.4 D43: SYSTEM_SETTING (reporting_currency, reporting_rate_source, reporting_rate_max_age_min, operating_timezone); system reporting CURRENCY_PAIRs with no project; REPORTING_VALUE view; invariant 38; CLIENT.exposure_ceiling stated in the reporting currency.

#	Question	Answer	In the model
25	Review point (v5.7): the ledger design — accrual timing, loss events, the NPL-GR reseller P&L, variance accounts, earnings kept unconverted and offset between partners, money in transit, fee arithmetic.	NPL (10–11 Oct): accrue shares and rebates at conversion; book a loss to NPL first and recover share by share; run NPL-GR's full reseller P&L inside NPLify with two invoices in EUR and the remittance markup absorbing partner cost and forex; one variance account per cause; earnings taken out of unconverted collections and offset between partners at the partner's rate of the day; sent money is not NPL's — the promise is discharged on release; bank transfers carry cents, cash rounds; fee parts on the principal; Jeton 1.25 %, Ali assumed 1 %; EUR → EUR 2.8 % split 50/50; NPL's profit = remittance + reseller earnings.	v5.7 D46: LEDGER_ACCOUNT purposes (VAR_ × 4, LOSS_RECEIVABLE, MAKEGOOD, LOSS_PAYABLE, RESELL_REVENUE, RESELL_COST, VENDOR_PAYABLE, RESALE_RECEIVABLE, REMIT_CLAIM, FX_RESULT; INTRANSIT retired), sources LossRecovery / Invoice / EarningsOffset, component resale; LOSS_RECOVERY and EARNINGS_OFFSET tables; INVOICE.kind, sender_id; EARNINGS_RECEIVABLE.recovered_by_offset_id; PARTNER_REBATE_RULE.payout_currency; CURRENCY_PAIR.amount_rounding exact; views RESELLER_MARGIN, REMITTANCE_MARGIN, NPL_PROFIT; invariants 16, 25, 26, 28, 39 restated, 40–44 new.

Appendix A · FRS reconciliation

A.1 Vocabulary

FRS term	Model entity
Client	CLIENT (the commercial relationship) — its arrangements are PROJECTs (D38; until v5.3 the FRS client mapped to PROJECT)
Project (NPL's 'Project Context': BF, BF Sub, Evo ...)	PROJECT under CLIENT — one arrangement with its own fees, partners and policies (D38)
Receiving Entity	RECEIVING_ENTITY (member of a counterparty RECEIVER, which belongs to a RECEIVER_GROUP) with its RECEIVING_ENTITY_ACCOUNTs
Sender	SENDER with SENDER_RECEIVER_ALLOW; its refund destination is a group-less sender_return RECEIVER
Partner (PT, Jeton)	PARTNER + PARTNER_CONFIG
Facilitating Entity / Wallet registry	COLLECTION_RECEIVING_ENDPOINT (kind wallet) under PARTNER_ENTITY for the partner's wallets; the FRS 'wallet registry' across all owners is the WALLET_REGISTRY view (D15, D40)
Rate-day versions	RATE_COMPARISON + PARTNER_RATE_VERSION
Settlement	SETTLEMENT + SETTLEMENT_LINE + CONFIRMATION (+ SETTLEMENT_RETURN when rejected)
Shortfall ledger	SHORTFALL + CL.<RECEIVER>.<CCY>.SHORTFALL
PT Comms / H Comms	CO.<PARTNER>.<CCY>.POOL (earnings receivable)
Inter-entity redirection	Offset across receiver groups: a counterparty paid_receiver_id in another group than the paying leg's
Double conversion / pass-through	DEAL_GROUP with two legs (funding_source = prior_leg), hop via a group-less partner_transit RECEIVER
Commission / rebate (Jeton 0.4 %, Aquanow 0.3 %)	PARTNER_REBATE_RULE (basis pct_of_amount or pct_of_partner_fee, scoped by sender or fee structure) + PARTNER_REBATE_ACCRUAL; income in EARN_REBATE (D27)
Markup shared with a customer (Raeen sub-accounts, downlines)	MARKUP_SHARE_RULE basis share_of_net_markup, party_kind client_party, settlement_mode party_retains; SHARE_RECEIVABLE (D26)
Fee by monthly volume (GDC)	FEE_TIER under FEE_STRUCTURE; DEAL_FEE_TIER portions; PROJECT_MONTH_VOLUME (D21)
Game reseller / NPL-GR	PROJECT with an own-desk PARTNER_CONFIG; OWN_WALLET as COLLECTION_RECEIVING_ENDPOINT; INVOICE paid in parts (D23, D24)
Customer-facing calculation ('market rate', 'fee 1 %')	FEE_STRUCTURE.customer_facing_fee_basis / pct; DEAL_GROUP.customer_facing_rate (D22)
Token (cash collection / settlement)	COLLECTION.token_ref, agent_contact_ref, counted_at; SETTLEMENT_LINE.token_ref; DEAL.cash_location; PARTNER_RATE_VERSION.location (D30)
Bank cutoff (2.30 pm GMT+8)	PARTNER.bank_cutoff_time + PARTNER_CUTOFF_OVERRIDE (extensions, early closes); PARTNER_PAIR.needs_bank_lock; DEAL.rate_locked_at, cutoff_at, conversion_due_date; SETTLEMENT.cutoff_at, execution_due_date (D29)
Wallet screening	WALLET_SCREENING; COLLECTION.sending_address; COLLECTION_RECEIVING_ENDPOINT.risk_band (D31)
Reversal of collection; cash short	LOSS_EVENT with its split; SHORTFALL for recoverable cash shortages (D32)
Holiday calendar	PARTNER.holiday_calendar (the calendar's name) + HOLIDAY rows (the dates); business-day arithmetic for due dates (D34)
Quote package	QUOTE_PACKAGE under DEAL_GROUP — one row per issue, hash = DEAL.quote_hash, sent_at starts validity (D35)

FRS term	Model entity
Message template	`MESSAGE_TEMPLATE` — versioned, global or per project, sender-facing fields only for quotes (D36)
Settlement (FRS) — was DISBURSEMENT	`SETTLEMENT`, `SETTLEMENT_LINE`, `SETTLEMENT_RETURN`; `PROJECT.settlement_policy` (D37)
Markup shared with a party — was INTRODUCER / REFERRAL_*	`MARKUP_SHARE_PARTY`, `MARKUP_SHARE_RULE`, `MARKUP_SHARE_ACCRUAL`; purpose `SHARE_PAYABLE` (D37)
Approved destination — was SETTLEMENT_REGISTRATION	`DESTINATION_APPROVAL` (D37)
Sender pays / receiver pays; lowest and highest markup; customer-facing calculation; fee on top of market / fee inside its rate; agent rate	`sender_pays_pct`, `receiver_pays_pct`; `min_markup_pct`, `max_markup_pct`; `customer_facing_fee_basis` / `_pct` / `_rate`; `partner_fee_pct` with `fee_on_market` / `fee_in_rate`; `rate_basis` market / agent_rate (D37)
Client balance at a partner — was CLIENT_BALANCE	`PROJECT_BALANCE(project, partner, currency)`; the client-level roll-up is `CLIENT_POSITION` (D38)
System / reporting currency	`SYSTEM_SETTING` reporting_currency + platform-scope `CURRENCY_PAIRs` (their `PAIR_RATE_SOURCE`) + `REPORTING_VALUE` view, an estimate at the latest market rate (D43, D44)
User roles (Operations, Finance, Management)	`ROLE` rows referenced by `USER.role_id`, with one `ROLE_RIGHT` row per role × controlled action (initiate / approve / escalate); rights per role in the Roles & Visibility Matrix (D44)

A.2 Deviations from the FRS (classified)

FRS statement	Model	Classification
P1 — no custody	No custody except under an approved reroute or in an own-desk project (NPL-GR); OWN_WALLET + REROUTE for contingency, invariant 3 and the custody view as the control	Decided (Q12 answered; D23, 9 October 2026)
P3 — partner books never mix	Accounts per partner never net; alternate partner on reroute; post-collection transfer only if Q3 confirms	Decided (Q12 answered: reroute with recover dues; partner accounts never net)
Strict Client → Entity → Sender, one parent each	RECEIVER_GROUP → RECEIVER → RECEIVING_ENTITY → account; SENDER with a group allow-list; FRS project = one default group	Generalisation per NPL instruction
Beyond grace → re-quote always	late_collection_policy per project; FRS project keeps requote	Per NPL instruction (Sept 2026)
Sender always priced on Kraken	FEE_STRUCTURE.rate_basis; FRS project = market	Generalisation
One settlement per deal, full amount	Accumulate → settle the balance rounded as the pair says (residual carried), or hold within the exposure cap where the project allows	Generalisation (D25)
Partner fixed at quote	Same, except under REROUTE	Consistent
Bank fee → shortfall → top-up	carry_forward is the FRS behaviour; waived / absorbed added	Per NPL instruction
Registry-only destinations	DESTINATION_APPROVAL.approval_status — FRS project has all approved	Generalisation
AQN-LT-Sub offset-settled collections	OWN_WALLET at Aquanow; pass-through is a two-leg deal group or a reroute via the own wallet; also NPL-GR's own-desk collection endpoint	Resolved (v4.0, D23)

Draft v5.7 — internal baseline. The Ledger Posting Design and its Reading Companion are at v5.7 with this draft; the Calculation Specification is at v5.7 too; the state machines, roles matrix, configuration schema, reading guide and fee outlines are still at their earlier editions and are to be regenerated against this version per §6.

Appendix B · Data dictionary (field annotations)

Every table also carries `created_at` and `created_by`. Types: int, string, bool, decimal, datetime, date. Rows changed in a revision are marked with the revision and decision that changed them.

B.1 Configuration

PROJECT

Field	Type	Key	Notes
id	int	PK	
client_id	int	FK	the client this arrangement belongs to (v5.4, D38)
name	string		
client_label	string		
quote_freshness_min	int		
quote_validity_min	int		
grace_min	int		
grace_drift_pct	decimal		
late_collection_policy	string		requote / honor_with_approval
bank_fee_treatment_default	string		waived / carry_forward / absorbed
receiver_fee_timing	string		at_conversion (default, Q6) / post_confirmation
network_fee_policy	string		absorbed / charged_back (project default; per-transaction override via FEE_DECISION). A network fee may or may not apply; when it does it is stated in USDT, when it does not it is zero and nothing posts (v5.7, D46)
settlement_policy	string		to_zero / hold_allowed (Q5)
shortfall_tolerance	decimal		
exposure_ceiling	decimal		cap on balance resting at a partner
status	string		

PARTNER

Field	Type	Key	Notes
id	int	PK	
name	string		Ali, Jeton, PT
disclosure_style	string		fee_in_rate / fee_on_market — whether the partner's fee sits inside its rate or is stated on top of market (D37; was markup_included / market_plus)
rate_expiry_time	string		
holiday_calendar	string		name of the source calendar (e.g. ID banking holidays); the dates are HOLIDAY rows (v5.3, D34)
supported_methods	string		crypto / bank / cash / local_deposit — the methods this partner offers (Ali: all four; Jeton and Aquanow: crypto, bank). Networks are implied by its wallet endpoints (v5.4, D39)
status	string		
bank_cutoff_time	time		nullable: the partner's standing daily bank cutoff for pairs that rely on the banking channel (Ali and other Asian partners, 14:30); empty for Jeton and Aquanow, which bind nothing. Governs locking the rate for same-day conversion and initiating an outgoing bank settlement; per-day changes in PARTNER_CUTOFF_OVERRIDE (v5.3 D29, v5.4 D39)
cutoff_timezone	string		IANA zone, e.g. Asia/Hong_Kong (v5.3, D29)

RATE_SOURCE

Field	Type	Key	Notes
id	int	PK	
code	string		KRAKEN / CMC / OANDA / XE
staleness_threshold_s	int		

MARKUP_SHARE_PARTY

Field	Type	Key	Notes
id	int	PK	
name	string		
status	string		

SENDER

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
name	string		Sender A, Sender B
risk_tier	string		
status	string		

RECEIVER_GROUP

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
name	string		
is_default	bool		exactly one per project
status	string		

RECEIVER

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
group_id	int	FK	receiver group; null unless kind = counterparty (v5.0, D13)
kind	string		counterparty / sender_return / partner_transit
sender_id	int	FK	when kind = sender_return
partner_config_id	int	FK	when kind = partner_transit
name	string		the group: Receiver X, Receiver Y
status	string		

OWN_WALLET

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
provider	string		e.g. Aquanow (Q10)
account_label	string		e.g. LT Sub
role	string		reroute_contingency / own_desk / collection_bridge — LT Sub at Aquanow is the NPL-GR own desk; NPL's ERC-20 wallet is a bridge for partners without that network (v5.4, D40)
networks	string		networks the wallet receives on (ERC-20 where Ali cannot) (v5.4, D40)
network	string		
address	string		
signer_arrangement	string		hardware / multisig
custody_max_hours	int		

THRESHOLD

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	

Field	Type	Key	Notes
kind	string		max_amount / settlement_limit / daily_slots / rate_variation_rel / custom
currency	string		
value	decimal		
condition_expr	string		custom conditions
escalation	string		finance / management

CURRENCY_PAIR

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	null when scope = platform (v5.4, D43/D44)
scope	string		project / platform — a platform pair (currency → reporting currency) belongs to no project, is managed under system settings, is priced through PAIR_RATE_SOURCE like any pair and is never quoted (v5.4, D44)
from_currency	string		
to_currency	string		
quote_direction	string		out_per_in / in_per_out
rate_precision	int		
amount_rounding	string		exact / truncate_unit / nearest_unit / round_up_unit / nearest_n / round_up_n (v5.3, D25; v5.7, D46: exact is the default for bank transfers — cents go out; the unit options apply where the pair says so, in practice to cash hand-overs)
cash_rounding	string		truncate_n / nearest_n / round_up_n — how the collected cash amount is rounded to cash_rounding_unit (v5.3, D25; was nearest only)
cash_rounding_unit	int		100 / 500 / 1000 by currency — the unit the collected cash amount is rounded to; AED cash 100, USD cash in Dubai 10 (NPL, 9 Oct)
status	string		
rounding_unit	decimal		required for nearest_n / round_up_n (1,000 for THB, INR, JPY payouts) (v5.3, D25)

PARTNER_CONFIG

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_id	int	FK	
applicable_for_reroute	bool		
status	string		
is_own_desk	bool		default false; NPL converts in its own wallet for this project (NPL-GR); enabling needs Management approval (v5.3, D23)

COLLECTION_SENDING_ENTITY

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
label	string		sender's own account or a client's account
account_ref	string		

SENDER_RECEIVER_ALLOW

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
receiver_group_id	int	FK	sendable_to (was receiver_id)

RECEIVING_ENTITY

Field	Type	Key	Notes
id	int	PK	
receiver_id	int	FK	
name	string		Entity X1, Entity X2
status	string		

PAIR_RATE_SOURCE

Field	Type	Key	Notes
id	int	PK	
currency_pair_id	int	FK	
rate_source_id	int	FK	
priority	int		

PARTNER_PAIR

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
currency_pair_id	int	FK	
partner_pricing	string		fee_on_market / fee_in_rate
partner_fee_pct	decimal		known only for fee_on_market
status	string		
needs_bank_lock	bool		the pair relies on the banking channel, so the partner's cutoff governs its rate lock and its settlement (Evo pairs into EUR; USDT → USD / SGD at Ali); crypto-only pairs do not (v5.3, D29)

SETTLEMENT_SENDING_ENTITY

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
narrative_name	string		what the receiver sees
status	string		

SETTLEMENT_RAIL

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
name	string		bank or EMI executing the transfer
bank_details_ref	string		
expected_bank_fee	decimal		
currency	string		

FEE_OVERRIDE

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
partner_pair_id	int	FK	
pct	decimal		
effective_from	datetime		
approved_by	int	FK	
sender_pays_pct	decimal		optional; with receiver_pays_pct replaces the structure's proportion (v5.3, D20)
receiver_pays_pct	decimal		optional; both blank → structure's proportion; both set → sum = pct (v5.3, D20)

RECEIVING_ENTITY_ACCOUNT

Field	Type	Key	Notes
id	int	PK	
receiving_entity_id	int	FK	
currency	string		
rail	string		SWIFT / SEPA / TRC-20 / ERC-20 / …
account_kind	string		bank / wallet (v5.0, D14)
bank_details_ref	string		external store; when account_kind = bank (was details_ref)
network	string		when account_kind = wallet
address	string		when account_kind = wallet; checksum-validated per network
status	string		
usage	string		standing / single_use — a local account the receiver gives for one payout (v5.3, D32)
reserved_for_deal_id	int	FK	nullable; the one deal a single_use account serves (v5.3, D32)

DESTINATION_APPROVAL

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
receiving_entity_account_id	int	FK	was receiving_entity_id
approval_status	string		pending_partner / approved / suspended
bank_details_ref	string		as paid by this partner; only when the account's kind is bank (v5.2, D16)
network	string		as registered with this partner; only when the account's kind is wallet (v5.2, D16)
address	string		as registered with this partner; only when the account's kind is wallet (v5.2, D16)
approved_at	datetime		
wallet_screening_id	int	FK	nullable; the receiver-side screening of a wallet account (v5.3, D31)

FEE_STRUCTURE

Field	Type	Key	Notes
id	int	PK	
partner_pair_id	int	FK	
rate_basis	string		market / agent_rate (D37; was market / partner)
mode	string		fixed / variable
pct	decimal		
sender_pays_pct	decimal		
receiver_pays_pct	decimal		
min_markup_pct	decimal		
max_markup_pct	decimal		
min_margin_pct	decimal		
variance_threshold_rel	decimal		
quoting_rate_policy	string		worst_applicable / designated / best_applicable
effective_from	datetime		
customer_facing_fee_basis	string		actual / sender_pays_on_agent_rate / fixed (v5.3, D22)
customer_facing_fee_pct	decimal		when customer_facing_fee_basis = fixed (BF via Ali shows 1 %) (v5.3, D22)

MARKUP_SHARE_RULE

Field	Type	Key	Notes
id	int	PK	
markup_share_party_id	int	FK	
project_id	int	FK	
partner_pair_id	int	FK	optional

Field	Type	Key	Notes
basis	string		share_of_earnings / fixed_pct / share_of_net_markup (v5.3, D26: base = fee parts – partner cost on the leg, on amount × market; rebates never in the base)
pct	decimal		
effective_from	datetime		
fee_structure_id	int	FK	optional scope: only deals priced on this structure (Raeen sub-account 50 %, Raeen direct downlines 75 %) (v5.3, D26)
party_kind	string		markup-share party / client_party (v5.3, D26)
settlement_mode	string		npl_pays / party_retains — the party holds the converted funds and pays NPL the remainder monthly (v5.3, D26)

PARTNER_REBATE_RULE

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
effective_from	datetime		
basis	string		pct_of_amount — the rebate is a percentage of the amount converted (Jeton 0.4 % of amount on a 1 % or 1.25 % fee, Novi 0.3 %, Aquanow 0.3 %); pct_of_partner_fee, retired by the D27 amendment, is dropped from the value set (D46 amendment)
pct	decimal		the rebate in the chosen basis (v5.3, D27)
fee_structure_id	int	FK	optional scope (v5.3, D27)
sender_id	int	FK	optional scope (Novi) (v5.3, D27)
payout_currency	string		the currency the partner pays the rebate in and the accrual is expressed in — the amount currency (v5.7, D46; pct_of_amount only, amendment)

PARTNER_ENTITY

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
name	string		the partner's legal vehicle: PT-tour, PT Global Inc
status	string		

COLLECTION_RECEIVING_ENDPOINT

Field	Type	Key	Notes
id	int	PK	
partner_entity_id	int	FK	the vehicle whose name is on the account or wallet
kind	string		bank / wallet
currency	string		
rail	string		nullable (v5.2, D18): a bank endpoint needs it to go active; optional for wallets, must match network when set
bank_details_ref	string		external store; when kind = bank
network	string		TRC-20 / ERC-20 / …; when kind = wallet
address	string		when kind = wallet; checksum-validated per network
status	string		
own_wallet_id	int	FK	nullable; set when the endpoint is NPL's own wallet — under an own-desk configuration (D23) or as a collection bridge for a partner that lacks the network or for risk isolation (D40); the endpoint still belongs to the leg's partner configuration
risk_band	string		standard / elevated — which screening decisions may land here (v5.3, D31)
usage	string		standing / single_use (ad hoc local deposit accounts) (v5.3, D32)
reserved_for_deal_id	int	FK	nullable; the one deal a single_use endpoint serves (v5.3, D32)
expected_amount	decimal		nullable; the amount a single_use endpoint was arranged for (v5.3, D32)

FEE_TIER

Field	Type	Key	Notes
id	int	PK	
fee_structure_id	int	FK	
min_monthly_volume	decimal		threshold in volume_currency, per calendar month
volume_currency	string		
pct	decimal		
sender_pays_pct	decimal		optional
receiver_pays_pct	decimal		optional
effective_from	datetime		

WALLET_SCREENING

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
subject_kind	string		sender_wallet / receiver_wallet
sender_id	int	FK	when subject_kind = sender_wallet
receiving_entity_account_id	int	FK	when subject_kind = receiver_wallet
network	string		
address	string		
provider_ref	string		screening tool reference
result	string		standard / elevated / unacceptable
decision	string		accept / accept_alternate / reject
screened_by	int	FK	Operations
screened_at	datetime		
evidence_ref	string		

PARTNER_CUTOFF_OVERRIDE

Field	Type	Key	Notes
id	int	PK	
partner_id	int	FK	
applies_on	date		one day; null for a weekday rule
weekday	string		recurring rule (Friday); null for a one-day override
cutoff_time	time		the cutoff that applies instead
kind	string		extension / early_close
reason	string		urgent request / pre-holiday / partner notice
partner_agreement_ref	string		the partner agreed (extensions)
recorded_by	int	FK	Finance

HOLIDAY

Field	Type	Key	Notes
id	int	PK	
partner_id	int	FK	
date	date		
name	string		Idul Fitri, Boxing Day
scope	string		bank_channel / partner
source	string		partner notice / public calendar

MESSAGE_TEMPLATE

Field	Type	Key	Notes
id	int	PK	

Field	Type	Key	Notes
project_id	int	FK	null = global default; a project row overrides it
code	string		quote / payment_instructions / confirmation_request / settlement_advice / cash_token
version	int		
channel	string		email / chat / portal / pdf
language	string		
body_ref	string		external content store
printed_fields	string		the model fields the template may print; sender-facing only for quote
effective_from	datetime		
approved_by	int	FK	Management
status	string		

CLIENT

Field	Type	Key	Notes
id	int	PK	
name	string		Betfair, Evo, 66 Group, GDC, NPL-GR
category	string		NPL's own grouping (Project Context tab: BF, Evo, Game Reseller ...)
legal_entity_ref	string		contracting entity, external store
kyc_ref	string		external store
relationship_owner	int	FK	USER
exposure_ceiling	decimal		optional cap on Σ balance resting at partners across the client's projects, stated in the reporting currency and compared through REPORTING_VALUE at the latest market rate (D38, D43)
status	string		

SYSTEM_SETTING

Field	Type	Key	Notes
id	int	PK	
key	string		reporting_currency / reporting_rate_max_age_min / operating_timezone / ... (the reporting rate source is the platform pair's PAIR_RATE_SOURCE, not a key)
value	string		
effective_from	datetime		
approved_by	int	FK	Management

B.2 Rates

MARKET_RATE

Field	Type	Key	Notes
id	int	PK	
rate_source_id	int	FK	
currency_pair_id	int	FK	
rate	decimal		
captured_at	datetime		

RATE_COMPARISON

Field	Type	Key	Notes
id	int	PK	
partner_pair_id	int	FK	
rate_date	date		
avg_spread_2w	decimal		
avg_spread_4w	decimal		
status	string		open / closed
created_at	datetime		created lazily by the first deal of the day

PARTNER_RATE_VERSION

Field	Type	Key	Notes
id	int	PK	
rate_comparison_id	int	FK	
version_no	int		
rate	decimal		normalised to CURRENCY_PAIR.quote_direction; derived from quoted_rate (v5.3, D28)
market_rate_id	int	FK	
spread_pct	decimal		
status	string		active / usable / breached / approved / rejected
confirmed_by	int	FK	
approval_reason	string		
expires_at	datetime		
quoted_rate	decimal		as the partner stated it (Ali: 1.1738 EUR→USDT style) (v5.3, D28)
quoted_direction	string		out_per_in / in_per_out; rate = quoted_rate, or 1 ÷ quoted_rate at the pair's precision when the directions differ — derived, never typed (v5.3, D28)
location	string		nullable; required on cash pairs — INR cash in Delhi and in Mumbai are two versions on one day (v5.3, D30)

B.3 Deal, collection, conversion, reroute, balance conversion

DEAL_GROUP

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
sender_id	int	FK	
receiver_group_id	int	FK	entitled group
client_ref	string		D-YYMMDD-seq, what the sender sees
from_currency	string		
to_currency	string		
sender_rate	decimal		
fee_structure_id	int	FK	
amount_in	decimal		
target_out	decimal		
entry_mode	string		quote_first / collect_first / estimate_first
state	string		
quoted_at	datetime		
customer_facing_fee_pct	decimal		stamped at quote (v5.3, D22)
customer_facing_rate	decimal		= sender_rate ÷ (1 – customer_facing_fee_pct); the 'market rate' the quote prints (v5.3, D22)
invoice_id	int	FK	nullable; the resale invoice this group collects (NPL-GR) — the obligation the deal pays (v5.3 D24, v5.7 D46)
invoice_amount	decimal		nullable; the part of the resale invoice this group collects, in the invoice currency — an advance, then the balance; Σ over the invoice's groups ≤ its amount (D46 amendment)
invoice_rate	decimal		nullable; payout currency per invoice currency at pricing (same direction as SETTLEMENT_LINE.discharge_rate): the stored base for the amount reserved per instalment and for the forex true-up (D46 amendment)

DEAL

Field	Type	Key	Notes
id	int	PK	
deal_group_id	int	FK	every deal belongs to a group
leg_no	int		

Field	Type	Key	Notes
leg_type	string		forward / return
funding_source	string		collection / returned_settlement / prior_leg
funding_ref_id	int		SETTLEMENT_RETURN id or prior DEAL id
project_id	int	FK	
sender_id	int	FK	
receiver_group_id	int	FK	attribution (D1)
receiver_id	int	FK	optional until settlement
partner_pair_id	int	FK	
partner_entity_id	int	FK	
amount_mode	string		target_out / fixed_in — fixed_in takes the fee off the principal, target_out adds it as a markup on the target (invariant 42, D46 amendment)
collection_method	string		crypto / bank / cash
cash_instructions	string		free text for the hand-over; the city is cash_location, the token is on the collection (D30)
fee_structure_id	int	FK	
market_rate_id	int	FK	the market snapshot the leg was priced on; equals the earlier leg's when rate_reused_from_deal_id is set (D45)
partner_rate_version_id	int	FK	
sender_rate	decimal		
amount_in	decimal		
gross_out	decimal		
client_net	decimal		
fee_sender_part	decimal		
fee_receiver_part	decimal		
priced_at	datetime		quote or conversion
sent_at	datetime		null on conversion-priced legs; = QUOTE_PACKAGE.sent_at of the package in force (v5.3, D35)
valid_until	datetime		end of validity as quoted; effective_valid_until is the one in force after extensions (D45)
grace_until	datetime		
effective_valid_until	datetime		the validity in force: valid_until as quoted, moved by each approved QUOTE_VALIDITY_EXTENSION (any number, no ceiling); grace_until follows it (v5.5, D45)
rate_reused_from_deal_id	int	FK	nullable; this leg was priced on the market rate and partner rate version of an earlier, expired leg at the sender's request — an approved override of the automated rate; the difference against the day's rate posts to VARIANCE (v5.5, D45)
rate_override_reason	string		why the earlier rate was honoured (v5.5, D45)
rate_override_approved_by	int	FK	Finance (v5.5, D45)
late_decision	string		none / honored / requoted
state	string		
quote_hash	string		hash of the quote package in force for the group (QUOTE_PACKAGE.content_hash) (v5.3, D35)
fee_override_id	int	FK	nullable; the override that priced this leg (v5.3, D20)
fee_tier_id	int	FK	nullable; base tier at quote (v5.3, D21)
tier_volume_basis	decimal		project month-to-date volume that justified the tier, stamped at quote (v5.3, D21)
rate_locked_at	datetime		when the partner confirmed the rate; before the cutoff → converts the same day, after → next business day (v5.3, D29)
conversion_due_date	date		the lock date when rate_locked_at precedes cutoff_at, else the next business day — weekends in the partner's cutoff_timezone and the partner's HOLIDAY rows skipped; Converting may not start earlier (v5.3, D29, D34)
conversion_deferred_reason	string		set when the due date is pushed because the next day's rate breaches the minimum margin (ask for a better rate, or wait a day) (v5.3, D29)

Field	Type	Key	Notes
cash_location	string		city of the cash hand-over; required on cash legs — rates and agents differ by city (v5.3, D30)
cutoff_at	datetime		the effective cutoff that applied on the lock day (standing time or override), stamped with rate_locked_at (v5.3, D29 amended 9 Oct)

COLLECTION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
collection_receiving_endpoint_id	int	FK	the partner endpoint paid into; null for cash (v5.1, D15; was endpoint_type + endpoint_id)
collection_sending_entity_id	int	FK	
amount	decimal		
evidence_ref	string		hash / slip / receipt
state	string		pending / verified / reversed (v5.3, D32); cash: token_issued → handed_over → counted → verified (D30). v5.7 (D46): after a loss the part stays verified — NPL's funds stand in — and is reversed only when Management unwinds the deal
verified_by	int	FK	
verified_at	datetime		
sending_address	string		crypto: the sender's declared wallet (v5.3, D31)
wallet_screening_id	int	FK	nullable; the screening that admitted the sending wallet (v5.3, D31)
screening_skipped	bool		projects that do not screen every sender up front (Evo) (v5.3, D31)
token_ref	string		cash: serial of the token (local-currency bill) that identifies the agent; the partner may replace the agent and the token before hand-over — the collection holds the current token, the audit log the previous one; the rate locked at quote is unchanged (v5.3 D30, v5.4 D41)
agent_contact_ref	string		cash: the receiving agent's contact, external store (v5.3, D30)
counted_at	datetime		cash: the partner counted and confirmed the funds (v5.3, D30)

CONVERSION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	null when owned by a balance conversion
balance_conversion_id	int	FK	null when owned by a deal leg
partner_rate_version_id	int	FK	
calc_mode	string		npl_calculates / partner_calculates
amount_in	decimal		
expected_out	decimal		
actual_out	decimal		booked figure, rounded as the pair says (D25)
rounding_residual	decimal		signed (v5.3, D25): positive = fraction dropped, stays client money; negative = amount added by rounding up, NPL's cost posted to EXP_ROUNDING
fee_receiver_part	decimal		taken at conversion (Q6)
variance	decimal		
residual_in	decimal		the margin kept unconverted in the collection currency (NPL's practice: earnings are taken out of money not yet converted); EARN_GROSS is then recognised in that currency and rests in NPL's pool at the partner in that currency (v5.7, D46)
basis_variance	decimal		
state	string		
approved_by	int	FK	

REROUTE

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
trigger	string		partner_unavailable / limit_reached / holiday (a HOLIDAY row of scope partner on the day) / compliance_hold / other
phase	string		pre_collection / post_collection
outcome	string		via_own_wallet / refund_sender (Q3, post-collection)
own_wallet_id	int	FK	
alternate_partner_config_id	int	FK	
forwarding_policy	string		forward_full / forward_and_recover_dues
recovered_amount	decimal		≤ outstanding receivable at the alternate partner
partner_agreement_ref	string		the partner agreed the net amount
reason	string		
requested_by	int	FK	
approved_by	int	FK	

SENDER_CREDIT

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		open / applied

EARNINGS_RECEIVABLE

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		outstanding / recovered
recovered_by_deal_id	int	FK	
recovered_by_offset_id	int	FK	nullable; the EARNINGS_OFFSET that moved this margin to another partner's unconverted balance (v5.7, D46)
recovered_at	datetime		

BALANCE_CONVERSION

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	
receiver_group_id	int	FK	group served; chosen split
from_currency	string		
to_currency	string		
partner_pair_id	int	FK	
partner_rate_version_id	int	FK	
market_rate_id	int	FK	
amount_in	decimal		
expected_out	decimal		
actual_out	decimal		

Field	Type	Key	Notes
variance	decimal		
rounding_residual	decimal		
state	string		
requested_by	int	FK	Finance
approved_by	int	FK	Management
converted_at	datetime		
rate_locked_at	datetime		(v5.3, D29)
conversion_due_date	date		banking-ready balances (USD, SGD) wait for the next business morning (v5.3, D29)

ENTITLEMENT_REATTRIBUTION

Field	Type	Key	Notes
id	int	PK	
balance_conversion_id	int	FK	
receiver_group_id	int	FK	
amount_from	decimal		
amount_to	decimal		

DEAL_FEE_TIER

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
fee_tier_id	int	FK	
amount_in_portion	decimal		Σ over a leg = amount_in
pct	decimal		the tier's pct at quote

INVOICE

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
kind	string		vendor (the vendor's invoice to NPL-GR at the preferential price) / resale (NPL-GR's invoice to its customer at retail price) (v5.7, D46)
vendor_invoice_id	int	FK	on a resale invoice: the vendor invoice it marks up (self-reference); RESELLER_MARGIN pairs the two through it (D46 amendment)
receiver_id	int	FK	the vendor owed, on a vendor invoice
sender_id	int	FK	the customer invoiced, on a resale invoice (v5.7, D46)
currency	string		
amount	decimal		
reference	string		
issued_at	datetime		
status	string		open / settled / void

QUOTE_PACKAGE

Field	Type	Key	Notes
id	int	PK	
deal_group_id	int	FK	
issue_no	int		1, 2, … per deal group
message_template_id	int	FK	the template version that rendered it
channel	string		email / chat / portal / pdf
rendered_ref	string		external document store
content_hash	string		= DEAL.quote_hash of the legs it prices

Field	Type	Key	Notes
printed_source_rate	decimal		= DEAL_GROUP.customer_facing_rate
printed_fee_pct	decimal		= DEAL_GROUP.customer_facing_fee_pct
printed_amount_out	decimal		
valid_until	datetime		= the leg's valid_until
sent_at	datetime		marks the package sent; starts validity
sent_by	int	FK	Operations
superseded_by_id	int	FK	the re-issue that replaced this one, if any

QUOTE_VALIDITY_EXTENSION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
extension_no	int		1, 2, … per leg
requested_at	datetime		
from_valid_until	datetime		
to_valid_until	datetime		
reason	string		the sender's request
approved_by	int	FK	Finance; Management past the cutoff or the rate version's expiry
approval_id	int	FK	

EARNINGS_OFFSET

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
from_partner_config_id	int	FK	where NPL's converted margin rests
from_currency	string		
from_amount	decimal		NPL's margin handed to the client there
to_partner_config_id	int	FK	where the client's unconverted money is
to_currency	string		
to_amount	decimal		= from_amount ÷ the rate used, taken from the client's unconverted balance
partner_rate_version_id	int	FK	the to-partner's rate of the day (NPL, 11 Oct: the partner's rate, not market)
market_rate_id	int	FK	the market snapshot of the day, for reference
requested_by	int	FK	Finance
approved_by	int	FK	Management
state	string		requested / approved / posted
posted_at	datetime		

B.4 Settlement, return, confirmation, fees, referrals, rebates

SETTLEMENT

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	
settlement_sending_entity_id	int	FK	
settlement_rail_id	int	FK	
currency	string		
total	decimal		rounded as the pair says (D25)
bank_fee_treatment	string		
state	string		

Field	Type	Key	Notes
prepared_by	int	FK	
approved_by	int	FK	
escalated_to	int	FK	
released_at	datetime		
cutoff_at	datetime		the effective cutoff of the approval day for a bank rail (standing time or override) (v5.3, D29 amended 9 Oct)
execution_due_date	date		the approval date when approved before cutoff_at, else the next business day (weekends and HOLIDAY rows skipped); a bank transfer is initiated no earlier (v5.3, D29, D34)

SETTLEMENT_LINE

Field	Type	Key	Notes
id	int	PK	
settlement_id	int	FK	
deal_id	int	FK	nullable; required when the paid receiver is group-less, optional on counterparty lines (v5.2, D17)
destination_approval_id	int	FK	
paid_receiver_id	int	FK	
amount	decimal		
receiver_fee_applied	decimal		exceptional; fee is normally taken at conversion
reference	string		entity invoice format
outcome	string		sent / confirmed / returned
invoice_id	int	FK	nullable; the invoice this line discharges — the vendor invoice for a vendor line, the resale invoice for the margin line to NPL-GR's own wallet (v5.3 D24, v5.7 D46)
obligation_discharged	decimal		in the invoice currency; reserved = obligation_discharged × DEAL_GROUP.invoice_rate, paid = obligation_discharged × discharge_rate; reserved – paid is the forex true-up posted to VAR_FX_TIMING against NPL's pool (D24, D46 amendment)
discharge_rate	decimal		payout currency per invoice currency at payment (v5.3, D24)
discharge_market_rate_id	int	FK	nullable; the market snapshot used (v5.3, D24)
token_ref	string		cash settlement: the receiver's token, provided up front and relayed NPL → partner → agent (v5.3 D30, v5.4 D41)
evidence_ref	string		cash settlement: the agent's photo of the receiver's token after hand-over, relayed partner → NPL → receiver; the receiver's CONFIRMATION closes the loop (v5.4, D41)

SETTLEMENT_RETURN

Field	Type	Key	Notes
id	int	PK	
settlement_line_id	int	FK	
returned_amount	decimal		
bank_fee	decimal		
reason	string		
returned_at	datetime		

CONFIRMATION

Field	Type	Key	Notes
id	int	PK	
settlement_line_id	int	FK	
amount_received	decimal		
slip_ref	string		
result	string		full / short
confirmed_by	int	FK	
confirmed_at	datetime		

SHORTFALL

Field	Type	Key	Notes
id	int	PK	
receiver_id	int	FK	
confirmation_id	int	FK	
amount	decimal		
state	string		open / topped_up
topped_up_settlement_id	int	FK	

BANK_FEE_EVENT

Field	Type	Key	Notes
id	int	PK	
settlement_id	int	FK	
expected_amount	decimal		
actual_amount	decimal		
treatment	string		waived / carry_forward / absorbed
state	string		open / applied
applied_settlement_id	int	FK	

FEE_DECISION

Field	Type	Key	Notes
id	int	PK	
subject_type	string		Deal / Settlement / BalanceConversion
subject_id	int		
fee_kind	string		markup / network / bank
decision	string		apply / waive / override
pct	decimal		
recommended_by	int	FK	Finance — reviews and recommends before Management approves a covered bank fee or a markup above the cap (v5.4, D41)
approved_by	int	FK	Management for a covered bank fee or a markup above max_markup_pct; Finance for the other one-off decisions
reason	string		

MARKUP_SHARE_ACCRUAL

Field	Type	Key	Notes
id	int	PK	
markup_share_rule_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		accrued / reconciled / settled (v5.3, D26; was accrued / paid)
paid_at	datetime		
direction	string		payable (NPL pays the party) / receivable (the party holds the funds and owes NPL its remainder) (v5.3, D26)
period	string		YYYY-MM; receivables reconcile monthly like rebates (v5.3, D26)
received_amount	decimal		on receivables (v5.3, D26)

PARTNER_REBATE_ACCRUAL

Field	Type	Key	Notes
id	int	PK	
partner_rebate_rule_id	int	FK	
deal_id	int	FK	

Field	Type	Key	Notes
expected_amount	decimal		
received_amount	decimal		
period	string		YYYY-MM
state	string		expected / reconciled / received
statement_ref	string		the partner statement line it reconciled against (v5.3, D27)

LOSS_EVENT

Field	Type	Key	Notes
id	int	PK	
side	string		collection / settlement
collection_id	int	FK	when side = collection
settlement_line_id	int	FK	when side = settlement
amount	decimal		≤ the verified or released amount
reason	string		account_frozen / cash_short / wrong_chain / other
occurred_at	datetime		
sender_share	decimal		
npl_share	decimal		
partner_share	decimal		
client_share	decimal		shares sum to amount
state	string		open / approved / posted
approved_by	int	FK	Management

LOSS_RECOVERY

Field	Type	Key	Notes
id	int	PK	
loss_event_id	int	FK	
party	string		sender / partner / client
amount	decimal		≤ the matching share on the LOSS_EVENT
state	string		agreed / received — a client share is received at once (it is a reversal of the make-good, never a payment)
received_at	datetime		
reference	string		the partner's credit, the sender's transfer, the client's agreement
approved_by	int	FK	Management (approve_loss_split)

B.5 Ledger & controls

LEDGER_TRANSACTION

Field	Type	Key	Notes
id	int	PK	
source_type	string		Collection / Conversion / BalanceConversion / Settlement / SettlementReturn / Confirmation / BankFeeEvent / Reroute / MarkupShare / Rebate / LossEvent / LossRecovery / Invoice / EarningsOffset / Adjustment (v5.7, D46: + LossRecovery, Invoice, EarningsOffset)
source_id	int		
description	string		
posted_at	datetime		
posted_by	int	FK	
reversal_of_id	int	FK	

LEDGER_POSTING

Field	Type	Key	Notes
id	int	PK	
ledger_transaction_id	int	FK	

Field	Type	Key	Notes
ledger_account_id	int	FK	
side	string		debit / credit
amount	decimal		
currency	string		
ownership_tag	string		client / company
cost_component	string		principal / earnings / rebate / partner_cost / bank_fee / network_fee / variance / share / rounding / fx_timing / cutoff_timing / loss / resale (v5.7, D46: + resale for NPL-GR's reseller books; network fees are stated in USDT and absent when none applies)
receiver_group_id	int	FK	nullable; set on every CL.PROJECT.<CCY>.PAYABLE line: the leg's group at collection and conversion, the paying leg's group on a release, return or short confirmation (SETTLEMENT_LINE.deal_id), the served group on a balance conversion, the deal's group on a loss share or an earnings offset — so GROUP_ENTITLEMENT is a sum over postings (D46 amendment, 11 Oct)

LEDGER_ACCOUNT

Field	Type	Key	Notes
id	int	PK	
code	string		OWNER.HOLDER.CCY.PURPOSE
ownership	string		client / company
holder_type	string		PartnerConfig (incl. the own desk) / OwnWallet / Receiver (also the vendor of VENDOR_PAYABLE) / Sender (also LOSS_RECEIVABLE, RESALE_RECEIVABLE) / MarkupShareParty / Project / None (v5.7, D46)
holder_id	int		
currency	string		
purpose	string		COLLECTED / HELD / DUE / PAYABLE / SHORTFALL / CREDIT / MAKEGOOD / POOL / WALLET / EARN_GROSS / EARN_REBATE / RESELL_REVENUE / RESELL_COST / EXP_PARTNER / EXP_BANKFEE / EXP_NETWORK / EXP_ROUNDING / EXP_LOSS / VAR_CONVERSION / VAR_CUTOFF / VAR_RATE_HONOUR / VAR_FX_TIMING / SHARE_PAYABLE / SHARE_RECEIVABLE / REBATE_RECEIVABLE / LOSS_RECEIVABLE / LOSS_PAYABLE / VENDOR_PAYABLE / RESALE_RECEIVABLE / REMIT_CLAIM / FX_RESULT (v5.7, D46: VARIANCE split into four; INTRANSIT retired — sent money is a line state)
kind	string		asset / liability / income / expense; FX_RESULT is income or expense by sign, per currency

USER

Field	Type	Key	Notes
id	int	PK	
email	string		Google Workspace
role_id	int	FK	ROLE (v5.4, D44; was a free-text role)
status	string		

APPROVAL

Field	Type	Key	Notes
id	int	PK	
subject_type	string		
subject_id	int		
action	string		
requested_by	int	FK	
approved_by	int	FK	never equal to requested_by
escalated_to	int	FK	
state	string		
reason	string		

AUDIT_LOG

Field	Type	Key	Notes
id	int	PK	
record_type	string		
record_id	int		
field	string		
old_value	string		
new_value	string		
actor_id	int	FK	
at	datetime		

RECORD_LOCK

Field	Type	Key	Notes
id	int	PK	
record_type	string		
record_id	int		
locked_by	int	FK	
locked_at	datetime		

ROLE

Field	Type	Key	Notes
id	int	PK	
code	string		operations / finance / management
name	string		
sees_economics	bool		false for operations: no derived-economics fields on any screen, export or message
status	string		

ROLE_RIGHT

Field	Type	Key	Notes
id	int	PK	
role_id	int	FK	
action	string		a controlled action: price_quote, extend_quote_validity, cross_group_payout, cover_bank_fee, enable_own_desk, approve_loss_split, override_screening_reject, earnings_offset (v5.7, D46), …
may_initiate	bool		
may_approve	bool		
escalates_to_role_id	int	FK	nullable; the role added on escalation